Backend mövzularını addım addım başa düşək

Bir kafe tətbiqi üzərindən doqquz mövzunu öyrənəcəyik. Müştəri menyunu açır, sifariş verir, server məlumatı saxlayır və cavab qaytarır. Bütün dərslər bu tanış hadisə ətrafında qurulub.

Hər hissə əvvəl problemi göstərir, sonra həllini izah edir. Yeni söz həmin problemə lazım olanda açılır. Kod nümunəsinin ardınca onun nə etdiyi və hansı nəticəni verdiyi göstərilir.

Oxu qaydası: əvvəl nümunəni başa düş, sonra qalın yazılmış nəticəni öz sözlərinlə de. Sonda verilən kiçik sualı cavablandır. Cavab alınmırsa bütün kitabı yenidən oxumaq əvəzinə həmin nümunəyə qayıt.

Kodlar öyrətmək üçün kiçik saxlanıb. Fraqment adlandırılan hissə mövcud layihənin müəyyən yerinə əlavə olunur. Göstərilməyən sinif və parametrlərin nə olduğu mətnin içində açıqlanır.

28 oktyabr 2026

Dərslərin ardıcıllığı

1 Sorğu serverdə necə işləyir 3

2 DI asılılığı kənardan verməkdir 8

3 EF Core əlaqəli məlumatı necə gətirir 16

4 Function və stored procedure nə ilə fərqlənir 25

5 İndeks məlumatı tapmağı necə dəyişir 32

6 Fraqmentasiya məlumatın yayılmasıdır 43

7 SOLID dəyişiklikləri nəzarətdə saxlamaq üçündür 50

8 Pattern təkrarlanan problem üçün həll quruluşudur 57

9 Async gözləyərkən threadi boş saxlamağa kömək edir 70

1 Sorğu serverdə necə işləyir

Kafedə müştəri telefondan menyunu açır. Ekranda məhsulların görünməsi üçün telefon serverdən siyahı istəyir. **Server** burada məlumatı hazırlayıb cavab verən proqramdır.

Bu dərsdə yalnız bir hadisəni izləyəcəyik: müştəri menyunu istəyir, server menyunu qaytarır. Həmin hadisənin başlanğıcdan sona qədər keçdiyi yol onun **həyat dövrüdür**. İngiliscə buna **lifecycle** deyilir.

Müştəri ilə server arasındakı yol

```
Telefon: "Menyunu ver."
             |
             v
Server: menyunu tapir ve cavab hazirlayir.
             |
             v
Telefon: gelen menyunu ekranda gosterir.
```

Telefonun göndərdiyi istək **request**, serverin qaytardığı cavab **response** adlanır. “Sorğu” və “cavab” deyəndə bunları nəzərdə tutacağıq.

Bu mərhələdə yadda saxla: server bir sorğunu qəbul edir, onun üçün iş görür və cavab qaytarır. Lifecycle həmin işin yolunu təsvir edir.

Sorğunun içində nə gəlir

Telefon serverə sadəcə “menyu” sözünü göndərmir. Sorğunun hansı işi istədiyini və hansı ünvana getdiyini bildirir. Məsələn:

HTTP sorğusunun sadələşdirilmiş görünüşü

```
GET /menu
```

HTTP müştəri ilə serverin sorğu və cavabı hansı qayda ilə göndərdiyini müəyyən edir. Yuxarıdakı `GET` məlumat oxumaq istədiyimizi bildirir. `/menu` hansı məlumatı istədiyimizi göstərən yoldur.

Serverdə `/menu` üçün yazılmış kod seçilir. Ünvanla onu işləyəcək kodun uyğunlaşdırılmasına **routing** deyilir. Seçilən işləyici nöqtəyə isə **endpoint** deyilir.

Server menyunu tapanda `200` statusu ilə cavab verə bilər. **Status kodu** cavabın nəticəsini bildirən rəqəmdir. Menyu tapılmırsa `404`, gözlənilməz server xətası varsa `500` ola bilər.

Beləliklə, ardıcıllıq budur: **ünvan gəlir, həmin ünvanın kodu seçilir, kod cavab hazırlayır.**

İlk oxunuşda yadda saxlamalı əsas əlaqə budur: `GET /menu` sorğudur; həmin sorğuya cavab verən kod endpoint-dir.

Sorğu niyə birbaşa menyu koduna getmir

Kafedə bəzi əməliyyatlar yalnız daxil olmuş istifadəçilər üçündür. Məsələn, “mənim sifarişlərim” ekranını açmazdan əvvəl istifadəçini tanımaq lazımdır.

Server buna görə əsas işi görməzdən əvvəl bir neçə yoxlamadan keçir. Bu yoxlamaları aparan komponentlərə **middleware** deyilir. Onların ardıcıllığı **pipeline**, yəni emal zənciridir.

Bir sorğunun nümunə yolu

```
Sorqu gelir
    |+    v
Xeta yaranarsa onu tutacaq qat
    |
    v
Istifadecinin kim oldugunun yoxlanmasi
    |
    v
Istifadecinin bu ise icazesinin yoxlanmasi
    |
    v
Sifarisleri getiren esas kod
```

Kimliyi müəyyən etməyə **authentication** deyilir: “Bu adam kimdir?” İcazəni yoxlamağa **authorization** deyilir: “Bu adam bu işi görə bilərmi?” Daxil olmuş müştərinin başqa müştərinin sifarişinə baxmağa icazəsi olmaya bilər.

Middleware sorğunu növbəti mərhələyə ötürə və ya özü cavab verib dayandıra bilər. İcazə yoxdursa əsas sifariş koduna getmədən cavab qaytarılır.

Nəticə: middleware əsas işə çatana qədər və iş bitəndən sonra görüləcək ortaq işləri təşkil edir.

Əvvəl və sonra kodunun sırası

İki middleware düşün. Birincinin adı A, ikincinin adı B-dir. Hər ikisi əvvəl bir qeyd yazır, növbəti mərhələni çağırır, sonra yenə qeyd yazır.

İşləmə sırası

```
A: evvel
    B: evvel
        Menyunu hazirlayan kod
    B: sonra
A: sonra
```

Sıra niyə belədir? A, B-ni çağırıb onun bitməsini gözləyir. B əsas kodu çağırıb onun bitməsini gözləyir. Əsas kod bitəndə əvvəl B-nin, sonra A-nın qalan hissəsi işləyir.

ASP.NET Core-da növbəti mərhələni təmsil edən parametr çox vaxt `next` adlanır. **ASP.NET Core** C# ilə veb server proqramı qurmaq üçün istifadə edilən çərçivədir. Çərçivə hazır server və sorğu emalı mexanizmlərini təqdim edir.

Əgər B növbəti mərhələni çağırmasa, əsas kod işləməz. Əgər əsas kod xəta ilə dayansa, “sonra” hissələrinin hamısının adi qaydada işləyəcəyinə güvənmək olmaz. Xətadan asılı olmayaraq görülməli təmizlik işi üçün `finally` bloku istifadə edilir. Bu blokun işi koddan çıxarkən təmizliyi yerinə yetirməkdir.

Yadda saxla: sorğu zəncirin içinə doğru gedir, uğurlu iş bitəndə davam kodu əks ardıcıllıqla tamamlanır.

Sorğu bitəndə nə bitir

Telefon menyunu aldı. Bu sorğu üçün iş bitdi. Amma server proqramı bağlanmır: növbəti müştərinin sorğusunu qəbul etməyə davam edir.

Burada iki müxtəlif ömür var:

Nəyin ömrü	Nə vaxt bitir
Bir HTTP sorğusu	Həmin sorğunun emalı tamamlananda.
Server proqramı	Tətbiq dayandırılanda və ya proses bağlananda.

Proses əməliyyat sistemində işləyən proqramın mühitidir. Eyni server proqramı çoxlu sorğunu emal edə bilər.

Sorğu üçün yaradılmış köməkçi obyektlərin də öz ömrü olur. Bəzisi yalnız bu sorğu üçün, bəzisi isə bütün server üçün paylaşılır. Bunun necə seçildiyini növbəti DI dərsində görəcəyik.

Müştəri əlaqəni kəsəndə server bunu ləğv siqnalı ilə bildirə bilər. Amma başladılmış hər iş özbaşına dərhal yox olmur. İşi görən kod siqnalı nəzərə almalıdır. Bunu async dərsində ayrıca açacağıq.

Özünü yoxla

Telefon eyni menyu ekranını iki dəfə yeniləsə nə olur? Adətən iki ayrı HTTP sorğusu göndərilir. Server proqramı isə həmin iki sorğu arasında işləməyə davam edir.

İcazə yoxlaması əsas kodu çağırmadan cavab verə bilərmi? Bəli. Middleware zənciri orada dayandıra bilər.

2 DI asılılığı kənardan verməkdir

Kafedə sifariş qəbul edən kod qəbz hazırlamalıdır. Əvvəlcə sifariş kodunun içində birbaşa qəbz hazırlayan obyekt yarada bilərik. Sabah qəbzi başqa formada hazırlamaq istəyəndə sifariş kodunu da dəyişməli olarıq.

DI bu bağlılığı azaltmaq üçün istifadə olunur. Sifariş kodu lazım olan qəbz hazırlayıcısını kənardan alır.

Dependency bir kod parçasının işləmək üçün ehtiyac duyduğu başqa obyektdir. Azərbaycan dilində “asılılıq” deyəcəyik. **Injection** həmin asılılığın kənardan verilməsidir. İkisi birlikdə **Dependency Injection**, qısa olaraq **DI** adlanır.

Məntiq

```
Sifaris kodunun isi: sifarisi qebul etmek.
Qebz kodunun isi: qebz hazirlamaq.

Sifaris koduna hazir qebz hazirlayicisi verilir.
```

İlk fikir budur: **bir sinif lazım olan hər obyekti özü yaratmağa məcbur deyil. Onu kənardan ala bilər.**

Bu fikri başa düşmək üçün hələ container və lifetime sözlərini bilməyə ehtiyac yoxdur.

Sinif obyekt və konstruktor eyni şey deyil

Sinif obyektin necə qurulacağını və nə edə biləcəyini yazdığımız tərifdir. **Obyekt** isə həmin tərifdən yaradılmış konkret nüsxədir.

C#

```
public class ReceiptPrinter
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

var printer = new ReceiptPrinter();
```

`ReceiptPrinter` sinifdir. `new ReceiptPrinter()` onun obyektini yaradır. `printer` həmin obyektə çatmaq üçün saxladığımız dəyişəndir. `var` dəyişənin tipinin sağdakı ifadədən müəyyən edilməsinə imkan verir.

`Print` bir metoddur, yəni çağırıla bilən əməliyyatdır. `string` nəticənin mətn olduğunu bildirir. `return` həmin nəticəni çağıran verir. `public` bu hissənin kənardan əlçatan olduğunu göstərir.

Konstruktor obyekt yaradılarkən onun ilkin vəziyyətini quran xüsusi metoddur. Növbəti səhifədə qəbz hazırlayıcısını məhz konstruktor vasitəsilə sifariş obyektinə verəcəyik.

Nüsxə sözünün ingiliscəsi **instance**-dir. “İki instance yarandı” deyiləndə iki ayrı obyekt yaradılması nəzərdə tutulur.

Asılılığı əvvəl içəridə sonra kənarda yaradaq

Əvvəl sifariş sinfi öz qəbz obyektini yaradır:

C#

```
public class OrderService
{
    private ReceiptPrinter printer = new ReceiptPrinter();

    public string Complete() => printer.Print();
}
```

Burada `OrderService` həm qəbz hazırlayıcısının konkret növünü seçir, həm də onu yaradır. `private` sahənin yalnız sinfin daxilində istifadə olunduğunu bildirir. => qısa metod yazılışıdır; bu nümunədə sağdakı nəticəni qaytarır.

İndi həmin obyekti kənardan verək:

C#

```
public class OrderService
{
    private ReceiptPrinter printer;

    public OrderService(ReceiptPrinter printer)
    {
        this.printer = printer;
    }

    public string Complete() => printer.Print();
}

var printer = new ReceiptPrinter();
var orders = new OrderService(printer);
```

`this.printer` obyektin içindəki sahədir; sağdakı `printer` konstruktora gələn parametrdir. Parametr çağırış zamanı verilən dəyərdir.

Bu artıq DI-dır. Qəbz hazırlayıcısı sifariş sinfinin içində yaradılmır. Onu konstruktorla veririk. Bunun üçün ayrıca kitabxana istifadə etmədik.

Interface dəyişə bilən hissəyə müqavilə verir

Əvvəlki nümunədə asılılığı kənardan verdik, amma sifariş sinfi hələ də konkret `ReceiptPrinter` tipini tanıyır. İndi qəbzin həm mətn, həm PDF variantı olduğunu düşün.

Sifariş koduna “qəbzi necə qurursan?” təfərrüatı lazım deyil. Ona qəbz hazırlamaq əməliyyatı lazımdır. **Interface** həmin əməliyyatı vəd edən müqavilədir.

C#

```
public interface IReceipt
{
    string Build();
}

public class TextReceipt : IReceipt
{
    public string Build() => "Metn qebzi";
}

public class PdfReceipt : IReceipt
{
    public string Build() => "PDF qebzi";
}
```

İki sinif də `Build` əməliyyatını təqdim edir. `: IReceipt` həmin müqavilənin yerinə yetirildiyini bildirir. Konkret həllə **implementation**, yəni implementasiya deyilir. Burada metodlar izah üçün mətn qaytarır; real PDF faylı yaratmır.

`OrderService` artıq konstruktorda `IReceipt` qəbul edə bilər. Ona `TextReceipt` də, `PdfReceipt` də vermək olar. Sifarişin əsas axını qəbz növünə görə dəyişməz.

DI interface olmadan da mümkündür. Interface-in burada əlavə etdiyi fayda **dəyişə bilən konkret həll üçün sabit müqavilə** yaratmasıdır.

Container bizim yerimizə obyektləri birləşdirir

Layihədə çoxlu sinif varsa hamısını əllə yaratmaq yorucu olur. .NET bunun üçün **DI container** təqdim edir. Container hansı tipin hansı obyektlə təmin olunacağını saxlayır və lazım olanda obyektləri birləşdirir.

C# qeydiyyat fraqmenti

```
builder.Services.AddScoped<IReceipt, TextReceipt>();
builder.Services.AddScoped<OrderService>();
```

Birinci sətir container-ə deyir: “IReceipt istənəndə TextReceipt ver.” İkinci sətir OrderService-i də container-in yarada biləcəyi xidmət kimi qeyd edir. `builder.Services` qeydiyyat siyahısıdır.

Servis burada proqramın istifadə etdiyi köməkçi obyektdir. Ayrı server və ya mikroservis mənasına gəlmir.

OrderService-in konstruktoru IReceipt istəyirsə, container əvvəl qəbz obyektini əldə edir, sonra onu OrderService-ə verir. Xidməti container-dən əldə etməyə **resolve** deyilir.

Qeydiyyat sətirinin işləməsi obyektin həmin anda mütləq yaradılması demək deyil. Obyekt çox vaxt ilk dəfə istənəndə yaranır.

IoC idarəetmənin başqa mexanizmə verilməsi barədə daha geniş anlayışdır. Bu nümunədə yaradılmanı container idarə edir. DI bunun asılılıqları verməyə aid üsuludur. SOLID dərsində isə DI ilə dizayn prinsipi olan DIP-in fərqini görəcəyik.

Transient scoped singleton fərqini iki sorğuda görək

Container-ə yalnız “hansı sinif?” demirik. “Nə vaxt yeni nüsxə yarat?” qaydasını da veririk. Bu qayda **lifetime**, yəni xidmətin paylaşılma ömrüdür.

Eyni xidmət birinci HTTP sorğusunda iki dəfə, ikinci sorğuda yenə iki dəfə istənilsin. Aşağıdakı hər hərf ayrı obyekti bildirir:

Lifetime	Birinci sorğu	İkinci sorğu
Transient	A, B	C, D
Scoped	A, A	B, B
Singleton	A, A	A, A

Transient: container-dən hər yeni istəmə zamanı yeni nüsxə alınır. Ancaq artıq aldığın obyektdə metod çağırmaq yeni resolve deyil.

Scoped: eyni scope daxilində eyni nüsxə alınır. **Scope** bir qrup xidmətin paylaşılma və təmizlənmə sərhədidir. ASP.NET Core adətən hər HTTP sorğusu üçün bir scope yaradır. “Hər istifadəçi üçün bir” və ya “hər thread üçün bir” demək deyil.

Singleton: eyni əsas container daxilində həmin qeydiyyatın bir nüsxəsi paylaşılır. Başqa server prosesinin ayrıca container-i və ayrıca singleton-u olur.

Seçimin əsası budur: **bu obyektin vəziyyətini kimlərlə və nə qədər müddət paylaşmaq istəyirəm?**

Uzunömürlü obyekt qısaömürlü obyekti saxlayanda

Kafedə bütün gün işləyən bir xidmət təsəvvür et. O, yalnız bir sifariş üçün nəzərdə tutulmuş iş dəftərini alıb bütün gün özündə saxlayır. Sonrakı sifarişlər də köhnə dəftərlə işləyir.

Singleton-un scoped obyekti konstruktorla alıb özündə saxlaması buna bənzəyir. **Captive dependency** qısaömürlü asılılığın uzunömürlü obyekt tərəfindən tutulmasıdır.

Problem

```
Singleton hizmeti: butun tetbiq boyu yasayir.
    |
    +-- bir sorqunun DbContext-ini sahesinde saxlayir.
```

DbContext EF Core-un baza ilə iş vahididir. O, yüklənmiş obyektləri və dəyişiklikləri izləyə bilər. Onu növbəti dərsdə açacağıq. Çox vaxt bir HTTP sorğusu üçün scoped qeydiyyatdan keçirilir. Eyni DbContext-i paralel işlərdə paylaşmaq təhlükəlidir.

İki əsas həll var. Xidmət həqiqətən sorğuya aiddirsə onu da scoped et. Xidmət uzunömürlü fon işidirsə, hər iş üçün ayrıca scope açıb işi həmin scope daxilində bitir.

`IServiceScopeFactory` ayrıca scope yaradan .NET xidmətidir. Açılmış scope-dan alınan DbContext-i iş bitəndən sonra singleton sahəsində saxlamırsan.

Əsas səbəb: qısaömürlü obyektin real istifadə müddəti nəzərdə tutulmuş sərhədi aşır.

Obyekti kim təmizləyir və DI nəyi həll etmir

Fayl və baza bağlantısı kimi resursları iş bitəndə buraxmaq lazımdır. **Dispose** həmin təmizləmə əməliyyatının adıdır. `IDisposable` bu əməliyyatı təqdim edən müqavilədir. `using` isə blokdan çıxanda Dispose çağırılmasını təşkil edir.

Container-in yaratdığı və təmizlənməsini idarə etdiyi scoped obyekt scope bitəndə təmizlənir. Singleton əsas container bitəndə təmizlənir. Container-dən aldığın eyni asılılığı hər istifadəçi ayrıca dispose etməməlidir.

Dispose obyektin yaddaşdan dərhal silinməsi deyil. **GC**, Garbage Collector, proqramın artıq çata bilmədiyi idarə olunan obyektlərin yaddaşını sonradan geri ala bilər. İdarə olunan burada yaddaşa .NET-in nəzarət etməsi deməkdir. Resursun buraxılması ilə yaddaşın geri alınması ayrı işlərdir.

DI yaradılma və əlaqələndirməni asanlaşdırır, amma aşağıdakı problemləri avtomatik həll etmir:

- **Dairəvi asılılıq:** A B-ni, B də A-nı tələb edir. Məsuliyyətləri yenidən bölmək lazımdır.
- **Gizli asılılıq:** biznes metodu container-dən özünə lazım olanı axtarır. Bu yanaşmaya Service Locator deyilir; constructor-dan real ehtiyaclar görünmür.
- **Paylaşılan vəziyyət:** singleton istifadəçilərin məlumatını ortaq sahədə saxlayırsa məlumat qarışa bilər.

Özünü yoxla

Bir sorğuda scoped xidmət iki yerdə istənilir. **Eyni obyekt alınır.** Həmin xidmət ikinci sorğuda istənilir. **Yeni scope üçün ayrı obyekt alınır.**

3 EF Core əlaqəli məlumatı necə gətirir

Kafedə sifarişlər ayrıca cədvəldə, sifarişə daxil olan məhsullar isə başqa cədvəldə saxlanır. **Cədvəl** eyni quruluşlu qeydlər toplusudur. Hər qeyd bir **sətir**, qeydin adı və məbləği kimi hissələri isə **sütun** adlanır.

Orders cədvəli	Müştəri	Məbləğ
42 nömrəli sifariş	7	30
43 nömrəli sifariş	7	10
44 nömrəli sifariş	9	20

42 nömrəli sifarişin daxilində çay və tort var. Bunlar OrderItems cədvəlində ayrı sətirlərdir. Hər item sətrində `OrderId = 42` yazılır ki, hansı sifarişə aid olduğu bilinsin. `Id` sifarişi fərqləndirən **əsas açar**, `OrderId` isə əlaqəni saxlayan **xarici açardır**.

EF Core baza məlumatı ilə C# obyektləri arasında işləməyi asanlaşdıran kitabxanadır. Bazada sifariş sətri, proqramda isə `Order` obyekti olur. Belə uyğunlaşdırma edən vasitələrə **ORM** deyilir.

Bu dərsin sualı budur: **sifarişi gətirəndə onun içindəki məhsulları da nə vaxt gətirməliyik?**

Order ilə order.Items arasındakı fərq

`Order` obyektinin `Id` və `Total` kimi adi məlumatları ola bilər. `Items` isə həmin sifarişin məhsullarına keçid verir.

C# ifadələrinin mənası

```
order.Id       // Sifarisin nomresi.
order.Total    // Sifarisin meblegi.
order.Items    // Sifarise daxil olan setirler.
```

Başqa əlaqəli obyektə və ya obyektlər siyahısına keçid verən xassəyə **navigation property** deyilir. `Items` çoxlu elementə, `Customer` isə bir müştəriyə keçid ola bilər. Xassə obyektin oxuya bildiyimiz və bəzən dəyişə bildiyimiz məlumat hissəsidir.

DbContext baza sorğularını göndərmək və lazım gəlsə obyekt dəyişikliklərini izləmək üçün istifadə etdiyimiz EF Core obyektidir. Nümunələrdə ona `db` deyəcəyik.

C#

```
var query = db.Orders.Where(o => o.Id == 42);
var order = await query.SingleAsync();
```

Birinci sətir sorğunu qurur. `Where` uyğun sətirləri seçən şərtdir. İkinci sətir onu icra edib bir nəticə istəyir. `SingleAsync` dəqiq bir nəticə gözləyir; sıfır və ya bir neçə nəticə xətadır. `await` nəticə hazır olana qədər asinxron gözləməyə imkan verir; bunu ayrıca async dərsində açacağıq.

Vacib nəticə: Order-un alınması Items-in də alındığını avtomatik sübut etmir.

Eager loading lazım olan əlaqəni əvvəlcədən seçir

Müştəri “sifarişin detalları” ekranını açır. Bilirik ki, sifarişlə yanaşı içindəki məhsullar da lazımdır. Ona görə hər ikisini əvvəlcədən tələb edirik.

C#

```
var order = await db.Orders
    .Include(o => o.Items)
    .SingleAsync(o => o.Id == 42);
```

`Include(o => o.Items)` “bu sifarişin Items əlaqəsini də gətir” deməkdir. Bu üsul **eager loading** adlanır. Məna etibarilə əlaqəli məlumatı əsas nəticə alınarkən hazır edirik.

İcra tamamlanandan sonra proqramda bu məlumat mövcuddur:

Nəticə

```
Order 42
    Items
        Cay
        Tort
```

İndi Items siyahısını gəzmək üçün həmin əlaqəni yenidən bazadan istəmək lazım deyil.

Əlaqənin davamını da seçmək olar. Məsələn, hər item-in Product məhsul məlumatı lazımdırsa `ThenInclude` həmin növbəti əlaqəni bildirir.

Eager loading-in mənası **əlaqəni əvvəlcədən istəməkdir**. “Həmişə bir SQL əmri olur” onun tərifi deyil. Sorğunun bir və ya bir neçə SQL əmrinə bölünməsinə bu dərsin sonrakı hissəsində baxacağıq.

Seçim: məlumatın lazım olacağını əvvəlcədən bilirsənsə eager loading aydın başlanğıcdır.

Explicit loading sonradan açıq əmr verir

İndi başqa ekran düşün. Əvvəl yalnız sifariş nömrəsi və məbləği görünür. Məhsullar istifadəçi “detalları aç” deyəndə lazımdır.

Əvvəl əsas sifarişi alırıq. Sonra ayrıca əmrlə Items-i yükləyirik:

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
```

`Entry(order)` həmin obyektin EF Core məlumatına çıxış verir. `Collection` çoxlu element saxlayan əlaqəni seçir. `LoadAsync` onu yükləyir. Tək müştəri əlaqəsi üçün `Collection` yerinə `Reference` istifadə olunur.

Bu üsul **explicit loading** adlanır. Explicit “açıq şəkildə göstərilmiş” deməkdir. Əlaqənin hansı sətirdə yükləndiyini koddan görə bilirik.

Bu sadə nümunədə birinci sifariş sorğusu, sonra məhsullar üçün əlavə sorğu işləyir. Üstünlük ehtiyac yarananda yükləməkdir. Xərc isə əlavə baza müraciətidir.

Burada yüklənmiş Order və həmin Order-la işləyən DbContext hələ istifadə edilə bilən vəziyyətdə olmalıdır. Context-in işini bitirib bağladıqdan sonra onunla yeni sorğu göndərə bilmərik.

Seçim: əlaqə bəzən lazımdırsa və əlavə müraciəti açıq idarə etmək istəyirsənsə explicit loading uyğundur.

Lazy loading əlaqəyə toxunanda yükləyir

Bu dəfə məhsulları açıq Load əmri ilə yükləmirik. Kod `order.Items`-ə çatanda EF Core, lazımdırsa, onları avtomatik gətirir. Bu üsul **lazy loading** adlanır.

Görünən kod ilə mümkün gizli iş

```
Gorunen: order.Items siyahisini oxu.

Arxada:
    Items hele yuklenmeyib?
        Bazaya sorgu gonder.
        Gelen setirleri Items-e yerlesdir.
```

Rahat tərəfi budur ki, ayrıca yükləmə çağırışı yazmırsan. Çətin tərəfi budur ki, adi xassə oxunuşu kimi görünən sətir bazaya gedə bilər.

EF Core-da lazy loading adi entity sinfində özbaşına aktiv olmur. **Entity** bazadakı qeydi təmsil edən obyektdir. Proxy üsulunda əlavə `Microsoft.EntityFrameworkCore.Proxies` paketi və `UseLazyLoadingProxies()` ayarı istifadə edilir.

Proxy xassəyə müraciəti tutub əlavə davranış edə bilən vasitəçi obyektdir. Bu üsulda uyğun əlaqələr `virtual` olmalıdır və entity sinfindən törəmək mümkün olmalıdır. `virtual` xassənin davranışının törəmə tipdə əvəz edilə bilməsini bildirir. `ILazyLoader` adlı başqa yanaşma da var; onun texniki tələbləri fərqlidir.

Yadda saxla: lazy loading asan yazılış verir, amma baza müraciətini kodda gizlədə bilər. Növbəti səhifə bunun niyə əhəmiyyətli olduğunu göstərir.

N+1 problemini üç sifarişlə sayaq

Üç sifarişin siyahısını aldıq. Sonra hər sifarişin hələ yüklənməmiş Items əlaqəsini ayrıca oxuduq. Lazy loading aktivdirsə belə nəticə yarana bilər:

Mərhələ	Bazaya müraciət
Sifariş siyahısını gətir	1-ci sorğu
42-nin məhsullarını gətir	2-ci sorğu
43-ün məhsullarını gətir	3-cü sorğu
44-ün məhsullarını gətir	4-cü sorğu

Bir ümumi sorğu və üç əlavə sorğu var. Ümumi say `1 + N` olur. Buna **N+1 problemi** deyilir. N burada sonradan əlaqəsi ayrıca yüklənən obyektlərin sayıdır.

100 sifariş üçün eyni davranış 101 sorğu yarada bilər. Dəqiq say əvvəldən hansı məlumatın yüklənməsindən də asılıdır. Əsas problem çoxlu əlavə gediş gəlişdir: hər müraciət serverlə baza arasında vaxt və resurs tələb edir.

Explicit Load əmrlərini dövrdə 100 dəfə çağırmaq da eyni çoxsaylı müraciət problemini yarada bilər. Sadəcə əlavə iş kodda açıq görünər.

Həll: ekranın ehtiyacını əvvəlcədən müəyyən et. Uyğundursa Include ilə əlaqəni gətir, lazım olan məlumatı bir sorğuda seç və ya əlavə məlumatı toplu al. Həqiqi sorğu sayını SQL əmrlərinin qeydlərindən yoxla.

Yalnız ad və say lazımdırsa bütün obyektləri gətirmə

Ekran yalnız sifariş nömrəsi, müştəri adı və məhsul sayı göstərir. Hər məhsulun tam obyektini yükləməyə ehtiyac yoxdur. Birbaşa nəticədə lazım olan məlumatı seçə bilərik:

C#

```
var rows = await db.Orders
    .Select(o => new
    {
        o.Id,
        CustomerName = o.Customer.Name,
        ItemCount = o.Items.Count
    })
    .ToListAsync();
```

`Select` nəticənin formasını qurur. Bu seçimə **projection** deyilir. `new { ... }` burada sadə nəticə obyekti yaradır. `Count` element sayıdır. `ToListAsync` nəticələri siyahı kimi alır.

EF Core bu ifadələri SQL-ə çevirə biləndə adı və sayı bazadan lazım olan formada alırıq. Əvvəl bütün məhsulları C# yaddaşına gətirib saymaq tələb olunmur. Bu nümunədə ayrıca Include yazmaq lazım deyil.

Şəbəkəyə qaytarmaq üçün ayrıca adlandırılmış sadə nəticə tipindən də istifadə etmək olar. Belə tipə çox vaxt **DTO**, yəni məlumatı ötürmək üçün obyekt deyilir.

Seçim: “tam əlaqəli entity lazımdır?” ilə “yalnız bir neçə nəticə dəyəri lazımdır?” suallarını ayır. Siyahı ekranında projection çox vaxt daha yığcam nəticə verir.

Tracking başqa suala cavab verir

Loading “əlaqəni nə vaxt gətirim?” sualıdır. **Tracking** isə “gətirdiyim obyektdə etdiyim dəyişiklikləri izləyimmi?” sualıdır.

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

order.Total = 35;
await db.SaveChangesAsync();
```

Tracking aktivdirsə EF Core obyektin əvvəlki və cari dəyərlərini nəzərə alır. `SaveChangesAsync` zamanı dəyişiklik bazaya yazıla bilər. İlkin dəyərlərin saxlanan surətinə **snapshot** deyilir.

Yalnız oxumaq istəyiriksə sorğuya `AsNoTracking()` əlavə edə bilərik. Bu halda həmin sorğunun entity-ləri əsas dəyişiklik izləyicisinə əlavə olunmur. Sonradan obyektin məbləğini dəyişmək və sadəcə SaveChanges çağırmaq həmin dəyişikliyi avtomatik yazmır.

No-tracking hər ssenaridə sabit faizlə daha sürətli deyil. Faktiki fayda nəticənin həcmi və təkrarlanan obyektlərdən asılıdır.

Eyni açarlı müştərinin nəticədə bir obyekt kimi təkrar istifadəsinə **identity resolution** deyilir. Əvvəl yüklənmiş uyğun obyektlərin əlaqələrinin avtomatik birləşdirilməsi **navigation fixup** adlanır.

Nəticə: eager ilə tracking, eager ilə no-tracking, projection və başqa kombinasiyalar mümkündür. Bunlar eyni seçim deyil.

İki kolleksiya niyə nəticəni böyüdür

Bir sifarişdə 2 məhsul və ayrıca 3 ödəniş qeydi olsun. SQL hər məhsulu hər ödənişlə eyni nəticədə birləşdirəndə həmin sifariş üçün `2 x 3 = 6` sətir yarana bilər.

JOIN əlaqəli cədvəllərin sətirlərini şərtə görə birləşdirən SQL əməliyyatıdır. Bir-birindən ayrı kolleksiyaların birləşməsi nəticəni vuraraq böyüdürsə buna **cartesian explosion** deyilir.

C#

```
var orders = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery()
    .ToListAsync();
```

`AsSplitQuery` uyğun əlaqə yükləməsini bir neçə SQL sorğusuna bölür. Böyük təkrarlanma azalır. Əvəzində əlavə baza gediş gəlişi və aralıq nəticələri yaddaşda saxlamaq lazım ola bilər. Dəqiq sorğu sayı quruluş və EF provider-indən asılıdır. **Provider** EF Core-u konkret baza sistemi ilə əlaqələndirən hissədir.

Ayrı sorğular arasında başqa istifadəçi məlumat dəyişə bilər. Hər nəticənin eyni anı əks etdirməsi tələb olunursa uyğun transaction və oxuma qaydası düşünülməlidir.

Son seçim

Əlaqə əvvəlcədən lazımdırsa eager; sonradan açıq alınacaqsa explicit; gizli alınması qəbul edilirsə lazy. Yalnız dəyərlər lazımdırsa projection seçimini düşün. Tracking-i isə dəyişiklik yazmaq ehtiyacına görə ayrıca seç.

4 Function və stored procedure nə ilə fərqlənir

Kafedə iki ayrı ehtiyacımız var. Birincisi, məbləğdən endirimi çıxıb cavab almaqdır. İkincisi, sifarişin vəziyyətini bazada dəyişməkdir. Birinci işin nəticəsi hesablanmış dəyərdir; ikinci iş bazada əməliyyatdır.

SQL Server-də **function** bir dəyər və ya cədvəl nəticəsini sorğunun içində istifadə etmək üçün verir. **Stored procedure** isə adla saxlanmış əmrləri icra edir. Hər ikisi bazada saxlanan təkrar istifadə olunan kod ola bilər.

SQL cədvəllərlə işləmək üçün dildir. `SELECT` məlumatı oxuyur, `INSERT` yeni sətir əlavə edir, `UPDATE` mövcud sətri dəyişir, `DELETE` sətri silir. SQL Server-də əlavə imkanları olan bu dil variantı **T-SQL** adlanır.

İki ehtiyac

```
"100-den 15 cix, cavabi ver." -> Function ucun numune.

"42-ci sifarisi tesdiqle."   -> Procedure ucun numune.
```

Bu dərsdəki function məhdudiyyətləri **SQL Server T-SQL funksiyalarına** aiddir. PostgreSQL kimi başqa bazalarda eyni qaydaları yoxlamadan tətbiq etmək olmaz.

Scalar function bir dəyər qaytarır

Endirimdən sonrakı məbləği hesablayaq:

T-SQL

```
CREATE OR ALTER FUNCTION dbo.NetAmount
(
    @Gross decimal(12,2),
    @Discount decimal(12,2)
)
RETURNS decimal(12,2)
AS
BEGIN
    RETURN @Gross - @Discount;
END;
GO

SELECT dbo.NetAmount(100, 15) AS Net;
```

Nəticə **85.00** olur. Tək nəticə dəyərinə **scalar** deyilir. `@Gross` və `@Discount` giriş parametrləridir: çağırış zamanı verilən dəyərlər. `decimal(12,2)` onluq tipdir; cəmi 12 rəqəm, onlardan 2-si vergüldən sonra ola bilər.

`RETURNS` nəticənin tipini, `RETURN` nəticənin özünü bildirir. `AS Net` cavab sütununa Net adını verir. dbo obyektin aid olduğu schema-dır. **Schema** baza obyektlərini ad altında qruplaşdırır.

`GO` SSMS kimi alətlərdə əmrlər qrupunu ayırır. Serverin adi T-SQL əmri deyil. Tətbiqdən göndərərkən həmin qruplar ayrı göndərilir.

Əsas fərq: function nəticəsini başqa SELECT ifadəsinin içində istifadə edə bilirik.

Procedure əmrləri adla icra edir

Orders cədvəlində `Id`, `CustomerId` və `Total` sütunlarının olduğunu qəbul edək. Müştərinin sifarişlərini gətirən procedure yazaq:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.GetOrders
    @CustomerId int
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;
END;
GO

EXEC dbo.GetOrders @CustomerId = 7;
```

`EXEC` procedure-u çağırır. `int` tam ədəd tipidir. `FROM` məlumatın hansı cədvəldən alınacağını, `WHERE` isə hansı sətirlərin seçiləcəyini bildirir. Bu procedure müştəri 7-nin sifariş cədvəlini qaytarır.

Qayıdan sətir və sütunlar toplusuna **result set** deyilir. Deməli, “procedure heç nə qaytarmır” düzgün izah deyil.

`SET NOCOUNT ON` təsirlənən sətir sayı barədə əlavə mesajları azaldır. SELECT nəticəsini gizlətmir. Onu hər sorğunu böyük faizlə sürətləndirən qayda kimi əzbərləmə.

Procedure SELECT edə bildiyi kimi INSERT, UPDATE və DELETE də edə bilər. SQL Server-də istifadəçinin yazdığı T-SQL function isə bazanın xarici vəziyyətini dəyişən belə iş vahidi üçün nəzərdə tutulmur.

Procedure nəticəni üç ayrı yolla verə bilər

Əvvəlki səhifədə sifariş cədvəlini SELECT ilə aldıq. Procedure ayrıca çıxış parametrini də doldura bilər:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.CountOrders
    @CustomerId int,
    @Count int OUTPUT
AS
BEGIN
    SELECT @Count = COUNT(*)
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    RETURN 0;
END;
GO

DECLARE @N int, @Status int;
EXEC @Status = dbo.CountOrders
    @CustomerId = 7, @Count = @N OUTPUT;
SELECT @N AS OrderCount, @Status AS Status;
```

`COUNT(*)` sətirləri sayır. `OUTPUT` procedure-un doldurduğu dəyəri çağırana çıxarır. `@N` say olur. `RETURN 0` isə bu nümunədə uğuru bildirən tam ədəd statusudur; `@Status` onu alır. `DECLARE` dəyişən elan edir.

Üç yolu ayır: **SELECT nəticə cədvəli verir; OUTPUT parametr dəyəri verir; procedure RETURN-u tam ədəd status verir.** Procedure RETURN-u function-un məbləğ qaytaran RETURN-u ilə eyniləşdirmə.

Table valued function cədvəl kimi istifadə olunur

Function yalnız bir rəqəm qaytarmağa məcbur deyil. **Table-valued function**, qısa **TVF**, cədvəl qaytarır. Onu başqa sorğunun məlumat mənbəyi kimi işlətmək olur.

T-SQL

```
CREATE OR ALTER FUNCTION dbo.CustomerOrders
(
    @CustomerId int
)
RETURNS TABLE
AS
RETURN
(
    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT Id, Total
FROM dbo.CustomerOrders(7)
WHERE Total > 20;
```

Function əvvəl müştərinin sifarişlərini müəyyən edir. Xarici sorğu isə nəticəni `Total > 20` ilə daraldır. Belə əlaqələndirilə bilmə function-un vacib üstünlüyüdür. Procedure normal SELECT-də bu cür birbaşa FROM mənbəyi kimi yazılmır.

Bir SELECT ilə yazılan bu formaya **inline TVF** deyilir. Baza onu əsas sorğu ilə birlikdə qiymətləndirə bilər.

Multi-statement TVF isə əvvəl nəticə cədvəlini dəyişən kimi yaradır, bir neçə əmrlə doldurur və qaytarır. Daha çox addım yazmaq mümkündür, amma planın seçilməsi çətinləşə bilər. Daha mürəkkəb forma avtomatik daha sürətli forma deyil.

Birlikdə tamamlanmalı dəyişikliklər transaction istəyir

Sifariş təsdiqlənəndə həm Orders vəziyyəti dəyişsin, həm məhsul ehtiyatı azalsın. Birinci dəyişiklik yazılıb ikinci uğursuz olsa, məlumat yarımçıq qalar.

Transaction birlikdə təsdiqlənməli baza işlərinin sərhədidir. `COMMIT` təsdiqləyir, `ROLLBACK` həmin transaction-un dəyişikliklərini geri alır. Bu birlikdəlik xüsusiyyətinə **atomiklik** deyilir.

Məntiqi axın

```
Transaction basla.
    Sifarisi tesdiqle.
    Ehtiyati azalt.

Her ikisi ugurludur -> COMMIT.
Xeta var           -> ROLLBACK.
```

Procedure transaction və xəta idarəetmə məntiqi daşıya bilər. T-SQL UDF, yəni istifadəçinin yaratdığı SQL Server function-u, bunun üçün uyğun iş vahidi deyil.

`TRY/CATCH` xətalı icranı tutmaq üçün bloklardır. `THROW` xətanı çağırana ötürür. Əsas fikir xəta gizlədib uğur demək əvəzinə nəticəni düzgün bildirməkdir.

Transaction bazadan kənara göndərilmiş email-i geri ala bilmir. Eyni qaydada başqa transaction-un içində çağırılan procedure-un daxili COMMIT-i müstəqil xarici təsdiq deyil. Transaction-un kim tərəfindən açıldığını bilmək lazımdır.

Seçim: bir-birinə bağlı baza dəyişikliklərini təşkil etmək üçün procedure və ya tətbiqin idarə etdiyi transaction düşün. Function-u nəticə ifadəsi kimi seç.

Performans və təhlükəsizlik barədə düzgün nəticə

Baza sorğunu icra etmək üçün yol seçir. Həmin yol **execution plan**, icra planıdır. Planın sonradan təkrar istifadə üçün saxlanmasına **plan cache** deyilir.

Procedure çox vaxt planı təkrar istifadə edə bilər. Amma plan yenidən qurula bilər və parametrli adi SQL də təkrar istifadədən yararlana bilər. Ona görə “procedure həmişə əvvəlcədən hazırdır və daha sürətlidir” doğru nəticə deyil.

Eyni function-u çox sətir üçün ayrı hesablatmaq da baha ola bilər. Bəzi SQL Server versiya və şərtlərində function əsas plana açıla bilər; buna **inlining** deyilir. Sürət barədə son qərar real plana və ölçməyə əsaslanır.

SQL injection istifadəçi mətninin məlumat əvəzinə SQL əmrinə çevrilməsidir. Procedure daxilində istifadəçi mətnini SQL-ə birləşdirmək bu riski yenə yarada bilər. Dəyərləri ayrıca parametr kimi ötürmək lazımdır.

Özünü yoxla

Hesablanmış məbləği SELECT sütunu kimi istəyirsən: scalar function və ya sadə ifadə düşün.

Sifariş nəticəsini başqa cədvəllə birləşdirmək istəyirsən: inline TVF uyğun ola bilər.

Sifarişi, ehtiyatı və audit qeydini dəyişmək istəyirsən: procedure və transaction sərhədini düşün. Audit baş vermiş əməliyyatın qeydidir.

5 İndeks məlumatı tapmağı necə dəyişir

Kafedə on minlərlə sifariş var. “Müştəri 7-nin sifarişlərini gətir” deyirik. Bazada uyğun axtarış quruluşu yoxdursa çoxlu sifariş səhifəsinə baxmaq lazım ola bilər.

Kitabda bir mövzunu axtarmağı düşün. Bütün səhifələri bir-bir oxumaq olar. Yaxud arxadakı göstəricidən mövzunun yerini tapıb həmin səhifəyə keçmək olar.

Baza **indeksi** həmin göstəriciyə bənzəyən ayrıca axtarış quruluşudur. Axtarılan dəyərlə məlumatın yerini əlaqələndirir.

İndeksin verdiyi fikir

```
Indeks yoxdur:
    Coxlu melumat sehifesine bax, uygun setirleri sec.

Uygun indeks var:
    Axtaris qurulusunda lazim olan hisseye cat.
    Uygun melumati oxu.
```

İndeksin məqsədi məlumatın özünü dəyişmək deyil, **uyğun sorğuda görülən oxuma işini azaltmaqdır**.

Amma kitabın yarısını oxumaq istəyirsənsə hər səhifəyə ayrıca göstəricidən getmək sərfəli olmaya bilər. Bazada da çoxlu nəticə qaytaran sorğu üçün başqa yol daha ucuz ola bilər.

Sütuna indeks verəndə nə yaranır

Bu SQL Server cədvəlini düşün:

Id	CustomerId	Total
42	7	30
43	7	10
44	9	20

T-SQL

```
CREATE INDEX IX_Orders_Customer
ON dbo.Orders(CustomerId);
```

Bu əmr `CustomerId` üzrə indeks yaradır. Sütunun içindəki 7 və 9 dəyərləri dəyişmir. Əlavə quruluş təxminən “müştəri 7 üçün bu sətirlərə, 9 üçün o sətrə bax” imkanı verir.

Sadələşdirilmiş xəritə

```
CustomerId 7 -> Order 42, Order 43
CustomerId 9 -> Order 44
```

Bu xəritə axtarış fikrini göstərir; real daxili quruluş bir qədər sonra izah olunur.

İndi `WHERE CustomerId = 7` şərti ilə sorğu gələndə baza bu indeksdən istifadə etməyi qiymətləndirə bilər. `WHERE` sətirləri seçən şərtdir. İndeksin mövcudluğu mütləq həmin indeksin seçilməsi demək deyil.

Yadda saxla: sütuna indeks vermək, o sütuna əlavə axtarış quruluşu yaratmaqdır. Hər sorğuya sürət zəmanəti vermək deyil.

B plus tree böyük siyahını hissələrə bölür

Adi SQL Server rowstore indeksləri **B+ tree** adlı balanslı ağac quruluşundan istifadə edir. Rowstore məlumatın sətir əsaslı saxlanmasıdır.

Ağacda yuxarıdan aşağı hər dəfə uyğun aralığı seçirik. Hər sətri əvvəldən yoxlamaq əvəzinə uyğun hissəyə yaxınlaşırıq:

Sadələşdirilmiş quruluş

```
Baslangic: [1..100] [101..200] [201..300]
                       |
                       v
                [101..150] [151..200]
                                |
                                v
                       [171, 172, 173, 174, ...]
```

Başlanğıc hissə **root**, aralıq hissə **branch**, son səviyyə **leaf** adlanır. Hər real qovşaqda çoxlu açar ola bilər. Bu, hər qovşağı yalnız iki hissəyə bölünən sadə binary ağac deyil.

Page, səhifə, bazanın məlumat saxladığı blokdur. SQL Server-in adi rowstore səhifəsi 8 KB-dır və içində bir neçə sətir ola bilər. Buna görə bir milyon sətirə baxmaq mütləq bir milyon disk oxunuşu demək deyil.

O(log n) böyümə barədə nəzəri ifadədir: məlumat artdıqca ağacda yolu tapmaq işi sətirlərin hamısını gəzmək qədər artmır. Bu ifadə dəqiq saniyə və disk oxunuş sayı vermir. Tapıldıqdan sonra nəticələrin özünü oxumağın da xərci vardır.

Clustered və nonclustered fərqi

İndi iki quruluşu ayıraq. Bu səhifə SQL Server-in adi sətir əsaslı indekslərinə aiddir.

Clustered index: ağacın yarpaq səviyyəsində cədvəlin məlumat sətirləri yerləşir. Yəni son səviyyəyə çatanda məlumatın özünə çatırıq. Bu səbəbdən cədvəldə bir clustered rowstore index olur.

Nonclustered index: ayrıca ağacdır. Yarpaqda indeks məlumatı və əsas sətri tapmağa kömək edən yer göstəricisi saxlanır.

Nonclustered indeksdən istifadə nümunəsi

```
CustomerId 7-ni indeksde tap.
    |
    v
Uyqun sifarisin yerini elde et.
    |
    v
Lazim olan diger sutunlari esas setirden oxu.
```

Yer göstəricisinə **locator** deyilir. Cədvəldə clustered index varsa nonclustered indeks clustered açardan istifadə edir. Clustered index olmayan SQL Server cədvəli **heap** adlanır; onda locator sətrin yerini başqa qayda ilə göstərir.

İndeksdən sonra çatışmayan məlumat üçün əsas sətrə getməyə **lookup** deyilir. Az nəticədə ucuz, çox nəticədə baha ola bilər.

Clustered index açar sırasına görə məntiqi quruluş yaradır. Bütün səhifələrin diskdə daim yan-yana olması və SELECT nəticəsinin avtomatik sıralı gəlməsi zəmanəti deyil. Cavab sırası üçün `ORDER BY` yazılır.

Primary key və unique indeks eyni sualı cavablandırmır

Primary key, əsas açar, cədvəldə hər sətri fərqləndirən qaydadır. Onun dəyəri təkrarlanmamalı və NULL olmamalıdır. NULL burada dəyərin olmadığını və ya naməlum olduğunu bildirən SQL işarəsidir.

Unique index müəyyən açarın təkrarlanmasına icazə vermir. Məsələn, sistemin qaydası email-in təkrarlanmamasıdırsa:

T-SQL

```
CREATE UNIQUE INDEX UX_Customers_Email
ON dbo.Customers(Email);
```

İki paralel əməliyyat eyni email-i yazmağa çalışanda unikal qaydanı baza özü qoruyur. Tətbiqdə “əvvəl yoxla, sonra yaz” demək təkbaşına bu zəmanəti vermir: iki əməliyyat da eyni anda “yoxdur” cavabı ala bilər.

Primary key məntiqi məlumat qaydasıdır. Clustered isə məlumatın saxlanma quruluşudur. SQL Server primary key üçün dəstək indeksi yaradır, amma primary key-in mütləq clustered olması tələb deyil.

Foreign key, xarici açar, başqa cədvəldəki uyğun açara istinad qaydasıdır. SQL Server-də belə qaydanın mövcudluğu həmin sütun üçün mütləq avtomatik əlavə indeks yaradıldığı demək deyil.

Nəticə: “nəyi unikal saxlamalıyam?”, “hansı əlaqə etibarlıdır?” və “sorğunu necə sürətləndirim?” əlaqəli, amma ayrı suallardır.

İki sütunlu indeksdə sıra niyə dəyişir

Telefon kitabçasında əvvəl soyad, sonra ad sırası olduğunu düşün. Soyadı biləndə uyğun hissəyə rahat gedirsən. Tək adı biləndə eyni ad müxtəlif soyadların arasında yayılır.

Composite index bir neçə sütunun birlikdə açar olduğu indeksdir. `(CustomerId, CreatedAt)` əvvəl müştəriyə, həmin müştərinin içində tarixə görə qurulur.

İndeksin məntiqi sırası

```
Musteri 7
    2026-10-20
    2026-10-21
Musteri 9
    2026-10-19
    2026-10-23
```

Müştəri 7-nin müəyyən tarixdən sonrakı sifarişləri bu quruluşa uyğundur. Yalnız tarixlə axtarışda isə həmin tarixlər müxtəlif müştəri hissələrində yayılır.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt);
```

“Ən çox fərqli dəyəri olan sütun həmişə əvvəl olmalıdır” qaydasını əzbərləmə. Əvvəl hansı sorğunun hansı bərabərlik və aralıq şərtlərindən istifadə etdiyini müəyyən et. Sonra sıralamanı və başqa sorğuları nəzərə al.

Yadda saxla: sütunların siyahısı qədər həmin siyahının sırası da indeksin hansı sorğuya rahat yol verdiyini dəyişir.

INCLUDE əsas sətrə əlavə gedişi azalda bilər

Sorğu müştəri və tarixlə axtarır, amma cavabda Total məbləğini də istəyir. İndeksdə Total yoxdursa baza onu əsas sətirdən ayrıca oxuya bilər.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt)
INCLUDE(Total);
```

`INCLUDE(Total)` məbləği də indeksin yarpaq səviyyəsində saxlayır. Amma Total axtarış açarlarının sırasına əlavə edilmir.

Bu sorğunun lazım olan məlumatı

```
Filtr:  CustomerId ve CreatedAt.
Cavab:  CreatedAt ve Total.

Hamisi bu indeksde tapila biler.
```

Bir sorğunun ehtiyac duyduğu bütün məlumat indeksdədirsə həmin sorğu üçün ona **covering index** deyilir. Covering “əhatə edən” deməkdir. Eyni indeks başqa, daha çox sütun istəyən sorğunu əhatə etməyə bilər.

Hər sütunu INCLUDE etmək də pulsuz deyil. İndeks böyüyür, daha çox yer tutur və həmin məlumat dəyişəndə əlavə iş yaradır.

Nəticə: açar sütunları axtarış və sıra üçün, INCLUDE sütunları isə nəticədə lazım olan əlavə məlumat üçün seçilir.

İndeks niyə bəzən seçilmir

Hansı yolu seçən hissəyə **optimizer** deyilir. O, sorğunu yerinə yetirmək üçün mümkün yolların xərcini qiymətləndirir.

Index seek uyğun açar aralığından başlamaqdır. **Index scan** indeksin böyük hissəsini və ya hamısını gəzməkdir. **Table scan** cədvəl məlumatını gəzməkdir.

Milyon sifarişin yalnız 3-ü lazımdırsa dar indeks axtarışı sərfəli ola bilər. Sifarişlərin 90 faizi lazımdırsa hər sətrə ayrıca lookup etməkdənsə geniş oxuma daha ucuz ola bilər. Seek həmişə yaxşı, scan həmişə pis deyil.

Selectivity şərtin məlumatın nə qədərini seçməsini ifadə edir. **Cardinality** kontekstdən asılı olaraq fərqli dəyər sayını və ya nəticə sətir sayını bildirə bilər.

Məsələn, iki statusdan yalnız biri cəmi 100 sətrə aiddirsə, həmin nadir hissə üçün indeks faydalı ola bilər. “Bool və az variantlı sütuna indeks həmişə mənasızdır” düzgün deyil.

Filtered index yalnız müəyyən şərtə uyğun sətirləri saxlayan SQL Server indeksidir. PostgreSQL-də oxşar anlayış partial index adlanır.

Optimizer-in dəyərlərin paylanmasını təxmin etməsinə kömək edən məlumata **statistics**, statistika deyilir. Səhv təxmin uyğun olmayan plan seçiminə səbəb ola bilər.

Oxuma qazancının yazma xərci var

İndeks cədvəllə uyğun qalmalıdır. Buna görə əlavə iş yaranır:

Dəyişiklik	İndeksdə görülən iş
INSERT	Yeni sətrə uyğun indeks məlumatı əlavə edilir.
İndeksdə saxlanan dəyərə UPDATE	Müvafiq indeks məlumatı yenilənir.
DELETE	Sətrin indeks məlumatı da silinmə baxımına düşür.

İndeks ayrıca disk sahəsi tutur. İstifadə olunan səhifələr yaddaşda yer tutur. Dəyişikliklərin bərpa üçün yazıldığı **transaction log** yükü də arta bilər.

Sütunun üzərində hesablama aparmaq axtarışı çətinləşdirə bilər:

T-SQL şərtləri

```
-- Ilk forma her tarixden il hesablayir:
WHERE YEAR(CreatedAt) = 2026

-- Ikinci forma tarix araligini birbasa gosterir:
WHERE CreatedAt >= '20260101'
  AND CreatedAt <  '20270101'
```

YEAR tarixdən ili çıxarır. İkinci forma uyğun indeks üçün daha rahat aralıq yarada bilər. İndeksdə səmərəli axtarış şərti kimi istifadə oluna bilən yazılışa **SARGable** deyilir. Bəzi hesablanmış sütun indeksləri kimi istisnalar da var; “funksiya varsa heç bir indeks işləməz” hökmü də doğru deyil.

PostgreSQL-də xüsusi indekslər nə üçündür

Bu səhifədəki növlər PostgreSQL üçündür. Onların adlarını SQL Server-in bütün cədvəllərinə eyni qayda ilə tətbiq etmək olmaz.

B-tree sıralana bilən dəyərlərdə bərabərlik və aralıq axtarışı üçün ümumi seçimdir. **Hash** dəyərdən hesablanan kodla sadə bərabərlik axtarışına xidmət edir; adi tarix aralığı sıralaması vermir.

GIN dəyərin içindəki hissədən onu saxlayan sətirlərə keçid yaradır. Məsələn, etiket siyahıları varsa:

GIN fikri

```
"cay" etiketi  -> 1, 4, 9 nomreli mehsullar.
"sirin" etiketi -> 2, 4 nomreli mehsullar.
```

GiST müxtəlif tiplərə uyğun axtarış qurmağa imkan verən çərçivədir. Həndəsi məlumatda bölgələrdən namizəd yerləri tapmaq nümunədir. **SP-GiST** məkanı hissələrə ayıran başqa quruluşları dəstəkləyir. Hansı müqayisələrin işləməsini konkret tip üçün operator qaydaları müəyyən edir.

BRIN hər sətir üçün ayrıca yer saxlamaq əvəzinə səhifə aralıqlarının xülasəsini saxlayır. Tarixlər fiziki sıra ilə yaxın gedirsə bəzi aralıqların sorğuya uyğun gəlmədiyini tez müəyyən edə bilər. Namizəd aralıqdakı real sətirlər yenidən yoxlanır.

Nəticə: indeks növünü məlumatın və axtarışın formasına görə seçirik. Bütün məlumat növlərinə bir indeks eyni dərəcədə uyğun deyil.

Düzgün indeks qərarını necə verək

Kafedə əsas sorğumuz belə olsun: müştərinin müəyyən tarixdən sonrakı sifarişləri, tarix və məbləğlə birlikdə göstərilsin.

T-SQL nümunəsi

```
SELECT CreatedAt, Total
FROM dbo.Orders
WHERE CustomerId = @CustomerId
  AND CreatedAt >= @From;
```

`@CustomerId` müştəri nömrəsi, `@From` başlanğıc tarix parametridir. `AND` hər iki şərtin doğru olmasını tələb edir. Bu sorğu üçün `(CustomerId, CreatedAt) INCLUDE(Total)` məntiqli namizəddir. Son qərar ölçmə ilə verilir.

SQL Server-də **execution plan** sorğunun hansı addımlarla icra olunduğunu göstərir. `SET STATISTICS IO ON` oxunan səhifələr barədə məlumat verir. **Logical read** səhifəyə yaddaş üzərindən müraciətdir. **Physical read** səhifəni diskdən gətirmək ehtiyacıdır; səhifə artıq yaddaşdadırsa yenidən disk oxunuşu lazım olmaya bilər.

PostgreSQL-də `EXPLAIN` planı, `EXPLAIN ANALYZE` isə sorğunu həqiqətən işlədərək icra məlumatını göstərir. Yazma sorğusunda ANALYZE də yazmanı icra edir.

Özünü yoxla

İndeks qoyandan sonra yalnız “seek göründü” demə. **Oxunan səhifə azaldımı, cavab vaxtı yaxşılaşdımı, əlavə yazma xərci qəbul edilirmi?** Bunları müqayisə et. Sabit “100 dəfə sürətlənəcək” vədi vermək olmaz.

6 Fraqmentasiya məlumatın yayılmasıdır

İndeks məlumatı səhifələrdə saxlayır. Vaxt keçdikcə yeni sətirlər əlavə edilir, bəzisi böyüyür, bəzisi silinir. Səhifələrin sırası və doluluğu dəyişə bilər.

Fraqmentasiya barədə düşünəndə iki ayrı sualı ayır:

- Açar sırası ilə getdiyim səhifələr faylda hansı ardıcıllıqla yerləşir?
- Eyni məlumat neçə səhifəyə yayılıb və bu səhifələr nə qədər doludur?

Eyni məlumat üçün iki fərqli vəziyyət

```
Yaxsi doluluq:
    [dolu sehife] [dolu sehife]

Asagi doluluq:
    [yari dolu] [yari dolu] [yari dolu] [yari dolu]
```

İkinci vəziyyətdə oxunacaq və yaddaşda saxlanacaq səhifə sayı arta bilər. Bu, məlumatın yanlış yazıldığı və ya itdiyi demək deyil.

Fraqmentasiya performans mövzusudur. Corruption isə məlumat quruluşunun korlanmasıdır. Bunları qarışdırma.

Bu dərsin əsas hissəsi SQL Server rowstore indekslərinə aiddir. Onların səhifələri və B+ tree məntiqi indeks dərsində izah edildi.

Xarici və daxili fraqmentasiyanı ayıraq

Açar aralığına görə səhifələr A, B, C sırası ilə oxunmalıdır. Amma həmin səhifələrin fayldakı nömrələri belə ola bilər:

Məntiqi sıra

```
A: acarlar 1..20    -> faylda sehife 10
B: acarlar 21..40   -> faylda sehife 90
C: acarlar 41..60   -> faylda sehife 11

Oxuma yolu: 10 -> 90 -> 11
```

Açar sırası ilə fiziki səhifə ardıcıllığının uyğun gəlməməsi **logical fragmentation** adlanır. Çox vaxt buna xarici fraqmentasiya deyilir. Böyük aralıqları oxuyanda ardıcıl gətirmənin səmərəsinə təsir edə bilər.

Page density səhifənin doluluğudur. Səhifə daxilində çox boş sahə qalmasına daxili fraqmentasiya kimi baxılır. Eyni məlumat daha çox səhifə tutursa yaddaş və oxuma xərci arta bilər.

Əməliyyat sisteminin baza faylını disk blokları arasında necə yerləşdirməsi isə başqa səviyyədir. **File system fragmentation** buna aiddir. SQL indeksini yenidən qurmaqla ümumi disk defraqmentasiyası eyni əməliyyat deyil.

Yadda saxla: sıra problemi ilə boşluq problemini ayrı ölç. Eyni “fraqmentasiya” sözü onların təsirini eyniləşdirmir.

Page split necə yaranır

Bir səhifədə 10, 20, 30, 40 açarlı sətirlər saxlanır. Səhifədə yeni sətir üçün yer qalmayıb. İndi 25 açarlı sətir əlavə etmək istəyirik.

Baza əlavə səhifə ayırıb sətirlərin bir hissəsini ora köçürə bilər. Bu bölünməyə **page split** deyilir.

Bölünmədən əvvəl

```
Sehife A: [10, 20, 30, 40]  Yer yoxdur.
```

```
Bölünmədən sonra sadələşdirilmiş görünüş
Sehife A: [10, 20]
Sehife B: [25, 30, 40]
```

Yeni səhifə faylın başqa yerində yerləşə bilər. İki səhifədə əlavə boşluq qala bilər. Bölünmə əlavə yazma və log işi yaradır. Silinmələr və böyüyən sətirlər də səhifə doluluğunu dəyişə bilər.

Fill factor indeks qurularkən yarpaq səhifələrini hansı hədəf doluluqla yaratmağı bildirir. Məsələn, 90 gələcək yazmalar üçün təxminən boş yer saxlayır. Sonradan həmin səhifənin daim 90 faiz dolu qalması demək deyil.

Aşağı fill factor bəzi split-ləri azalda bilər. Amma başlanğıcdan daha çox səhifə yaradır. Buna görə bütün indekslərə kor şəkildə aşağı dəyər vermək düzgün deyil.

Nəyi ölçmədən qərar verməməliyik

SQL Server indeksin səhifə sayı, sıra uyğunsuzluğu və doluluğu barədə məlumat verə bilər:

T-SQL diaqnostika nümunəsi

```
SELECT
    page_count,
    avg_fragmentation_in_percent,
    avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(), OBJECT_ID(N'dbo.Orders'),
    NULL, NULL, 'SAMPLED'
)
WHERE index_level = 0
  AND alloc_unit_type_desc = 'IN_ROW_DATA';
```

`page_count` səhifə sayıdır. `avg_fragmentation_in_percent` sıra uyğunsuzluğu göstəricisidir. `avg_page_space_used_in_percent` doluluq göstəricisidir. `index_level = 0` yarpaq səviyyəsini seçir.

`DB_ID()` cari bazanı, `OBJECT_ID` Orders obyektini müəyyən edir. `SAMPLED` nümunə götürməklə analiz aparır. `IN_ROW_DATA` sətir daxilində saxlanan məlumat hissəsidir. Sorğunun görünməsi üçün uyğun baza icazələri lazımdır.

Tək faizə baxma. Beş səhifəli indeksdə yüksək faiz praktik olaraq böyük problem olmaya bilər. Milyonlarla səhifəyə yayılmış boşluq isə daha əhəmiyyətli ola bilər.

Qərarın əsası: ölçü, doluluq və real sorğunun gördüyü iş birlikdə qiymətləndirilir. Diaqnostika özü də resurs sərf edir; hədəf cədvəldən başlamaq faydalıdır.

Reorganize və rebuild eyni iş üsulu deyil

Defraqmentasiya quruluşu daha əlverişli formaya gətirmək işidir. SQL Server-də iki əsas indeks baxımı üsulu var:

Üsul	Nə edir
REORGANIZE	Mövcud indeksin yarpaq səhifələrinin sırasını və sıxlığını mərhələli yaxşılaşdırır.
REBUILD	İndeksi yenidən qurur. Daha böyük resurs və əlavə sahə ehtiyacı yarada bilər.

T-SQL alternativ əmrlər

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REORGANIZE;

ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REBUILD;
```

Bunlar bir-birinin ardınca mütləq işlədiləcək siyahı deyil, iki alternativ nümunədir.

Reorganize indeks statistikasını ayrıca yeniləmir. Rebuild müvafiq indeks statistikasını yeniləyə bilər, amma cədvəlin bütün ayrı sütun statistikalarını avtomatik düzəltmir. Statistika dəyərlərin paylanması barədə plan seçiminə kömək edən məlumatdır.

Online baxım işləyərkən istifadəçi girişini daha çox davam etdirməyə imkan verir. Bütün kilidlərin yox olması demək deyil. Dəstək versiya, indeks növü və məhsul paketindən asılıdır. **Offline** baxım daha çox bloklanma yarada bilər.

Nəticə: resurs xərci ilə gözlənən sorğu qazancını müqayisə etmədən bütün indeksləri rebuild etmə.

Hansı vəziyyətdə baxım fayda verə bilər

Sifariş hesabatı geniş tarix aralığını oxuyur və çoxlu səhifə gəzir. İndeksdə aşağı doluluq varsa daha sıx quruluş səhifə sayını azalda bilər. Burada baxımın real faydası ola bilər.

Başqa sorğu isə bir sifariş nömrəsini dərhal tapır. Yüksək fraqmentasiya faizi həmin dar axtarışın mütləq yavaş olduğunu göstərmir.

Köhnə şablonlarda 5 və 30 faiz kimi hədlər görünə bilər. Bunlar bütün sistemlər üçün dəyişməz qanun deyil. **Əvvəl yavaş sorğunu ölç, sonra problemin fiziki quruluşdan gəlib gəlmədiyini yoxla.**

Saxlama qurğusu da təsir edir. SSD elektron yaddaş əsasında, HDD isə mexaniki disk əsasında işləyir. Səhifələrin fiziki sırasının təsiri eyni olmaya bilər. Aşağı doluluğun daha çox səhifə və yaddaş xərci yaratması isə yenə qalır.

Columnstore sətirləri başqa qayda ilə, sütun əsaslı sıxılmış qruplarda saxlayan indeks ailəsidir. Orada silinmiş sətir payı və qrupların ölçüsü kimi başqa göstəricilər əhəmiyyətlidir. Rowstore fraqmentasiya faizini ora olduğu kimi tətbiq etmə.

Özünü yoxla

Rebuild-dən sonra faiz yaxşılaşdı, sorğu vaxtı dəyişmədi. **Bu mümkündür.** Yavaşlığın əsas səbəbi başqa yerdə ola bilər. Məqsəd tək faizi deyil, real işi yaxşılaşdırmaqdır.

PostgreSQL-də VACUUM və REINDEX fərqi

PostgreSQL bir sətir dəyişəndə köhnə versiyanı dərhal yox etməyə bilər. Çünki həmin versiyanı hələ oxuyan başqa iş ola bilər. Bir neçə sətir versiyası ilə paralel işi idarə etməyə **MVCC** deyilir.

Sadələşdirilmiş dəyişiklik

```
Evvel: Order 42, Total = 30.
Sonra: Order 42, Total = 35.

Kohne oxucu ucun 30 versiyasi hele lazim ola biler.
Yeni oxucu yeni qaydalara uygun 35-i gore biler.
```

Artıq heç bir aktiv işə lazım olmayan köhnə sətir versiyasına **dead tuple** deyilir. Tuple burada sətir versiyasıdır. Bu və başqa boşluqlar faydalı məlumatla müqayisədə saxlanmanı böyüdə bilər; artıq şişməyə **bloat** deyilir.

VACUUM təmizlənə bilən köhnə versiyaların yerini gələcək istifadə üçün azad edir. Adi VACUUM ümumən bütün faylı sıxıb boşluğu diskə qaytarmır. **ANALYZE** plan üçün statistikanı yeniləyir. **Autovacuum** bu işləri avtomatik aparan mexanizmdir.

VACUUM FULL cədvəli yenidən yazıb sıxlaşdıra bilər, amma güclü kilid və əlavə sahə tələb edir. **REINDEX** indeksin yeni quruluşunu yaradır. Bunlar bir-birinin adı dəyişmiş eyni əməliyyat deyil.

Yadda saxla: PostgreSQL versiya təmizliyi ilə SQL Server rowstore sıra fraqmentasiyasını eyniləşdirmə.

7 SOLID dəyişiklikləri nəzarətdə saxlamaq üçündür

Kafe tətbiqi böyüyür. Yeni endirim növü, başqa qəbz formatı, başqa baza və yeni bildiriş kanalı əlavə olunur. Hər kiçik dəyişiklik bütün proqramı pozmamalıdır.

SOLID bu problemi düşünmək üçün beş dizayn prinsipinin baş hərfləridir. Dizayn burada kodun hansı hissəsinin hansı işi gördüyünü və hissələrin bir-birinə necə bağlı olduğunu müəyyən etməkdir.

Hərf	Əsas fikir
S	Bir-birindən ayrı dəyişmə səbəblərini qarışdırma.
O	Seçilmiş davranışı mövcud əsas axını dağıtmadan genişləndir.
L	Yeni növ köhnə müqavilənin vədini pozmasın.
I	İstifadəçini lazım olmayan metodlardan asılı etmə.
D	İş qaydasını dəyişən texniki detala möhkəm bağlama.

Bu cədvəli əzbərləmək hələ prinsipləri bilmək deyil. Növbəti səhifələrdə hər prinsipin hansı dəyişiklik problemini həll etdiyini görəcəyik.

Məqsəd: daha çox sinif və interface yaratmaq yox, dəyişikliyin hara təsir edəcəyini aydınlaşdırmaqdır.

S eyni sinfi niyə müxtəlif adamlar dəyişir

Bir InvoiceService qəbzin vergisini hesablayır, onu bazaya yazır, PDF görünüşünü qurur və email göndərir.

Sabah vergi qaydası dəyişir. Sonra dizayner qəbzin görünüşünü dəyişir. Sonra baza texnologiyası dəyişir. Hamısına görə eyni sinif dəyişirsə ayrı səbəblər bir yerə yığılıb.

SRP, Single Responsibility Principle, əlaqəli bir dəyişmə səbəbinə xidmət edən məsuliyyət sərhədi qurmağı istəyir.

Mümkün bölgü

```
InvoiceCalculator -> Meblegi ve vergini hesablayir.
InvoiceStore      -> Bazada saxlayir.
InvoiceRenderer   -> Qebzin gorunusunu hazirlayir.
InvoiceSender     -> Hazir qebzi gonderir.
```

İndi qəbzin rəngi dəyişəndə vergi hesabı dəyişməyə bilər. Baza dəyişəndə görünüş koduna toxunmaya bilərik.

Bu işləri ardıcıllaşdıran ayrıca xidmət də ola bilər: hesabla, saxla, göndər. Addımları əlaqələndirmək özü bir məsuliyyət ola bilər.

SRP “hər sinifdə bir metod olsun” demək deyil. Vergi hesablayan sinifdə bir neçə əlaqəli köməkçi metod normaldır. Əsas sual həmin metodların eyni səbəbə görə birlikdə dəyişib dəyişməməsidir.

Bir yerin içindəki hissələrin eyni məqsədə xidmət etməsinə **cohesion**, uyğunluq deyilir. SRP uyğun olmayan dəyişiklikləri ayırmaqla bunu yaxşılaşdırmağa kömək edir.

O yeni endirim əsas sifariş axınını dəyişməsin

Normal müştəriyə 10, VIP müştəriyə 20 faiz endirim veririk. Sonra tələbə endirimi əlavə olunur. Hər yeni növ üçün Checkout-un içində uzun seçim məntiqi artırsa əsas axın da daim dəyişir.

Endirim hesabını ayrıca müqaviləyə çıxaraq:

C#

```
public interface IDiscount
{
    decimal Calculate(decimal total);
}

public class VipDiscount : IDiscount
{
    public decimal Calculate(decimal total)
        => total * 0.20m;
}
```

`decimal` dəqiq onluq ədəd tipidir. `0.20m` həmin tipdə 20 faiz dəyəridir. `Calculate` endirimin öz məbləğini qaytarır; yekun ödəniş `total - endirim` olur.

Checkout konstruktorla IDiscount alsın. Yeni StudentDiscount eyni müqaviləni yerinə yetirsə Checkout-un hesab axını dəyişmədən yeni variantı istifadə edə bilər.

Bu **OCP**, Open Closed Principle, üçün nümunədir: seçilmiş dəyişiklik istiqamətində genişlənməyə imkan veririk, sabit əsas məntiqə az toxunuruq.

Qeydiyyat və seçim yerinin dəyişməsi normaldır. OCP “heç bir fayl heç vaxt dəyişməsin” demir. İki sabit hal üçün sadə `switch` də kifayət edə bilər; hər seçimə dərhal ayrıca pattern qurmaq məcburi deyil.

L yeni növ eyni vədi saxlamalıdır

Bir müqaviləmiz var: “etibarlı mətni verilən yerə yaz”. Proqram bu müqaviləyə güvənərək yazma metodunu çağırır.

İndi yalnız oxumağı bacaran obyekt həmin yazma müqaviləsini götürür və bütün çağırışlara “mən yaza bilmirəm” xətası verir. Adı və metod imzası uyğun olsa da davranış vədi pozulub.

LSP, Liskov Substitution Principle, yeni alt növün baza müqaviləsinin yerində istifadə ediləndə onu istifadə edən kodun düzgün gözləntilərini pozmamasını istəyir.

Müqavilə nəyi deyir

```
Qebul edilen giris: etibarli metn ve yazila bilen yol.
Ved edilen netice: metn yazilir ve ya ved edilen xeta gelir.

Yanlis yeni davranis:
    Etibarli girisde de "hec vaxt yaza bilmirem".
```

Çağırışdan əvvəl tələb olunan qaydaya **precondition**, uğurlu çağırışdan sonra vəd edilən nəticəyə **postcondition** deyilir. Obyektin etibarlı vəziyyətində qoruduğu qayda isə **invariant** adlanır.

Müqavilədə disk dolanda xəta ola biləcəyi deyilibsə, həmin xətanın baş verməsi təkbaşına LSP pozuntusu deyil. Əsas məsələ gözlənilməyən yeni məhdudiyyət və ya zəiflədilmiş vəddir.

Praktik həll: yalnız oxuyan obyektə yazma vədi yükləmə. Müqaviləni onun həqiqətən bacardığı əməliyyatlara uyğun seç. Növbəti prinsip bu bölgünü açır.

I hər istifadəçiyə lazım olan müqaviləni ver

Kafedə bir hesabat xidməti yalnız sifariş oxuyur. Amma ona verilən IOrderStore müqaviləsi oxuma, əlavə etmə, silmə, arxivləmə və başqa işlərlə doludur.

Hesabat xidməti istifadə etmədiyi yazma əməliyyatlarına da bağlı olur. Yalnız oxuyan başqa implementasiya isə yalandan yazma metodları təqdim etməyə məcbur qala bilər.

C# müqavilə nümunələri

```
public interface IOrderReader
{
    string Read(int id);
}

public interface IOrderWriter
{
    void Save(string order);
}
```

Hesabat IOrderReader alır. Yazma xidməti IOrderWriter alır. Ehtiyac varsa bir konkret sinif hər iki müqaviləni həyata keçirə bilər. `void` metodun nəticə dəyəri qaytarmadığını bildirir. Bu kiçik nümunədə sifariş mətnlə təmsil edilir.

Bu **ISP**, Interface Segregation Principle, üçün nümunədir. Burada “client”, müştəri, interface-dən istifadə edən kod deməkdir; kafedəki insan deyil.

ISP “hər interface-də yalnız bir metod olsun” demir. Birlikdə istifadə olunan oxuma metodlarını eyni müqavilədə saxlamaq normal ola bilər.

Nəticə: interface-i onu istifadə edən kodun real ehtiyacına görə böl. Sadəcə xırdalamaq üçün bölmək də əlavə qarışıqlıq yarada bilər.

D iş qaydası konkret baza sinfinə bağlanmasın

Sifarişi tamamlamaq biznes işidir. SQL bağlantısını açmaq isə həmin işin texniki detalıdır. Sifarişin əsas məntiqi birbaşa konkret SQL kitabxanasının təfərrüatına bağlanarsa texnologiya dəyişəndə əsas məntiq də dəyişə bilər.

İşin ehtiyacını bir müqavilə ilə göstəririk:

Asılılıq istiqaməti

```
Sifaris isi -> IOrderStore <- SqlOrderStore

Sifaris isi saxlamani teleb edir.
SqlOrderStore hemin telebi SQL ile yerine yetirir.
```

Bu **DIP**, Dependency Inversion Principle, üçün nümunədir. İş qaydası çox vaxt **high-level**, onu yerinə yetirən texniki detal **low-level** adlandırılır. Hər ikisini uyğun abstraksiya ilə bağlayırıq. Abstraksiya burada “sifarişi saxla” ehtiyacını ifadə edən müqavilədir.

DIP ilə DI fərqi: DIP dizaynda nəyə bağlı olduğumuzu seçir. DI isə seçilmiş asılılığı obyektə kənardan vermək üsuludur. Interface yaratmadan da constructor-la konkret SQL sinfi vermək mümkündür; bu DI-dır, amma texniki detaldan asılılığı aradan qaldırmır.

Qatların “controller service-i çağırır” formasında düzülməsi təkbaşına DIP zəmanəti deyil. Müqavilənin özü də konkret SQL texnologiyasının tipləri ilə doludursa detal yenə əsas işə sızmış ola bilər.

Beş prinsipi eyni dəyişiklik üzərində yoxla

Yeni qəbz formatı əlavə edirik. Özümüzdən bunları soruşaq:

S: görünüş dəyişdiyi üçün vergi və saxlanma siniflərini də dəyişirəmmi? Edirəmsə məsuliyyət sərhədi qarışmış ola bilər.

O: yeni qəbz implementasiyası əlavə edib mövcud sifariş axınını saxlayırammı?

L: yeni implementasiya eyni qəbul edilən girişlər üçün eyni vədi yerinə yetirirmi? Yoxsa köhnə kod indi əlavə xüsusi hallar yazmağa məcburdur?

I: qəbzi yalnız göstərən kodu çap və silmə kimi lazım olmayan əməliyyatlardan asılı etmişəmmi?

D: sifariş işi qəbz müqaviləsini bilir, yoxsa konkret PDF kitabxanasının iç quruluşunu?

Bir dəyişikliklə məşq

Əvvəl TextReceipt verən sifariş axını qur. Sonra PdfReceipt əlavə et. Sifariş kodunda hansı dəyişikliklərin lazım olduğunu yaz. Məqsəd interface sayını artırmaq yox, yeni variantın təsir etdiyi yerləri azaltmaqdır.

Yadda saxla: SOLID düzgün proqramı avtomatik yaradan düstur deyil. Dəyişiklik səbəbi, müqavilə və asılılıq istiqaməti barədə düşünmək üçün vasitədir. Əlavə abstraksiya problemi asanlaşdırmırsa onun xərci də nəzərə alınmalıdır.

8 Pattern təkrarlanan problem üçün həll quruluşudur

Kafe tətbiqində qəbzin mətn və PDF variantı, çatdırılmanın adi və sürətli variantı, ödənişin müxtəlif üsulları yarandı. Hər yerdə eyni dəyişiklik problemlərini təkrar görürük.

Design pattern belə təkrarlanan problemə verilən ümumi həll quruluşudur. Hazır köçürüləcək kod deyil. Onun adını bilməzdən əvvəl həll etdiyi problemi başa düşmək lazımdır.

Dəyişən hissəni əvvəl seç

```
Hansi davranis deyisir?       -> Strategy dusun.
Hansi obyekt alinmalidir?     -> Factory dusun.
Movcud interfeys uygun deyil? -> Adapter dusun.
Eyni ise elave davranis?      -> Decorator dusun.
```

Bu adlar seçimə kömək edir. Eyni quruluşu hər problemə tətbiq etmək lazım deyil.

Klassik **GoF** kataloqunda 23 pattern var. GoF həmin kitabın dörd müəllifinə verilən addır. Pattern-lər üç ailəyə ayrılır: **creational** yaradılmanı, **structural** quruluşu, **behavioral** davranış və ünsiyyəti təşkil edir.

Bu dərsdə əvvəl backend-də asan rast gəlinən problemləri açacağıq. Sonra qalan pattern-ləri konkret nümunə ilə bir-birinə bağlayacağıq.

Strategy dəyişən hesabı ayrıca saxlayır

Adi müştəri və VIP üçün fərqli endirim hesablayırıq. Məqsəd hər ikisində eynidir: endirim məbləğini tapmaq. Dəyişən hissə hesablama qaydasıdır.

SOLID dərsindəki `IDiscount` nümunəsini xatırla. Hər variant `Calculate(total)` təqdim edir. Əsas sifariş kodu seçilmiş variantı çağırır:

C# istifadə fraqmenti

```
decimal discountAmount = discount.Calculate(total);
decimal payable = total - discountAmount;
```

`discount` constructor-la verilmiş IDiscount obyektidir. `discountAmount` endirim məbləği, `payable` ödəniləcək məbləğdir.

Bu quruluş **Strategy** pattern-dir. Əsas kod eyni məqsədli alqoritmlərdən birini istifadə edir. **Alqoritm** nəticəyə çatmaq üçün addımlar qaydasıdır.

Əlaqə

```
Checkout -> IDiscount
                |
                +-- NormalDiscount
                +-- VipDiscount
                +-- StudentDiscount
```

Yeni endirim gələndə onun hesabı ayrıca yazılır. Checkout-un əsas hesablaması dəyişməyə bilər. Hansı strategiyanın seçilməsi isə ayrıca seçim yerinin işidir.

Nə vaxt uyğundur: eyni işi görən qaydaların variantları artır və müstəqil dəyişir. Cəmi iki sabit, sadə hal varsa bir şərt ifadəsi də kifayət edə bilər. Pattern-in əlavə quruluş xərci də var.

Factory hansı obyektin alınacağını bir yerə toplayır

Müştəri qəbzi PDF istəyir, kassir isə mətn qəbzi istəyir. Hər istifadə yerində “PDF-dirsə bunu yarat, mətndirsə onu yarat” kodunu təkrarlamaq istəmirik.

Factory obyekt yaratma və ya uyğun obyekti vermə məntiqini ayrıca yerə toplayır. Sadə seçim metoduna çox vaxt **Simple Factory** deyilir.

Sadə factory fikri

```
CreateReceipt("text") -> TextReceipt obyekti.
CreateReceipt("pdf")  -> PdfReceipt obyekti.
```

Klassik **Factory Method** bir qədər fərqlidir: baza sinif ümumi işi saxlayır, hansı məhsul obyektinin yaradılacağını alt sinfin əvəz edə bildiyi metoda verir.

Factory Method əlaqəsi

```
ReportJob: "Exporter yarat, sonra export et."
    |
    +-- CsvJob CreateExporter metodunu deyisir.
    +-- PdfJob CreateExporter metodunu deyisir.
```

Baza sinif ortaq davranışı təqdim edən sinifdir. **Alt sinif** ondan törəyib müəyyən addımı dəyişən sinifdir. Export məlumatı başqa formata çıxarmaqdır. CSV sadə cədvəl mətn formatıdır.

Hər adı Create olan metod GoF Factory Method deyil. Əsas məntiq yaradılma qərarının alt sinfə verilən dəyişmə nöqtəsi olmasıdır.

Strategy ilə fərq: Strategy davranışın necə hesablanmasına, Factory isə hansı obyektin yaradılıb verilməsinə aiddir. Birlikdə işləyə bilərlər.

Abstract Factory uyğun məhsul ailəsi yaradır

Kafe proqramının açıq və tünd görünüşü var. Hər görünüşün düyməsi və seçim qutusu bir-birinə uyğun olmalıdır.

Bir factory yalnız bir obyekt yox, uyğun bir neçə obyekt yarada bilər:

Məhsul ailəsi

```
LightFactory
    CreateButton()   -> aciq duymə.
    CreateCheckbox() -> aciq secim qutusu.

DarkFactory
    CreateButton()   -> tund duymə.
    CreateCheckbox() -> tund secim qutusu.
```

Bu **Abstract Factory** pattern-dir. Məqsəd uyğun məhsullar ailəsini birlikdə yaratmaqdır. Onu sadəcə “factory-lərin idarəçisi” kimi əzbərləmək əsas məntiqi gizlədir.

Yeni görünüş ailəsi əlavə etmək rahatlaşır. Amma ailəyə yeni məhsul növü, məsələn slider, əlavə etsən bütün factory-lərin müqaviləsi dəyişə bilər. Slider dəyəri sürüşdürərək seçən görünüş elementidir.

DI ilə əlaqə: container obyektlərin əlaqələrini və lifetime-ını qura bilər. Factory isə konkret girişə görə uyğun variantın alınmasını aydın ifadə edə bilər. .NET DI da seçim funksiyası və açarlı xidmətlər kimi üsullar təqdim edir. “DI işləmə vaxtında seçim edə bilmir” düzgün iddia deyil.

Əsas sual ad deyil: bir obyekt seçirsən, alt sinfə yaradılma addımı verirsən, yoxsa uyğun məhsul ailəsi yaradırsan? Bu fərq factory növünü başa salır.

Adapter uyğun gəlməyən çağırışı çevirir

Köhnə printer kitabxanasında metod `SendText` adlanır. Yeni sifariş kodumuz isə `IPrinter.Print` müqaviləsi gözləyir. İş eynidir, giriş formaları uyğun deyil.

Adapter bu uyğunlaşdırmanı ayrıca obyektə yığır:

C#

```
public class PrinterAdapter : IPrinter
{
    private LegacyPrinter oldPrinter;

    public PrinterAdapter(LegacyPrinter oldPrinter)
    {
        this.oldPrinter = oldPrinter;
    }

    public void Print(string text)
    {
        oldPrinter.SendText(text);
    }
}
```

Bu fraqmentdə IPrinter-in `void Print(string text)` müqaviləsini, LegacyPrinter-in isə `SendText(string text)` metodunu təqdim etdiyini qəbul edirik. Legacy köhnədən qalmış sistem deməkdir.

Sifariş kodu IPrinter-i tanıyır. Adapter çağırışı köhnə metodun formasına çevirir. Köhnə kitabxananı bütün tətbiqə yaymırıq.

Uyğunlaşdırma yalnız metodun adını dəyişmək olmaya bilər. Məsələn, yeni müqavilə manat, köhnə sistem qəpik qəbul edirsə məbləğ və yuvarlaqlaşdırma da düzgün çevrilməlidir.

Yadda saxla: Adapter-in əsas niyyəti fərqli müqavilələri uyğunlaşdırmaqdır.

Decorator eyni müqaviləyə əlavə davranış verir

Qiymət xidmətimiz işləyir. İndi hər qiymət istəniləndə bir qeyd yazmaq istəyirik. Əsas qiymət hesabını dəyişmədən onu başqa obyektlə sarıya bilərik.

C#

```
public class LoggedPrice : IPrice
{
    private IPrice inner;

    public LoggedPrice(IPrice inner)
    {
        this.inner = inner;
    }

    public decimal Get(int id)
    {
        Console.WriteLine("Qiymet istendi");
        return inner.Get(id);
    }
}
```

IPrice müqaviləsində `decimal Get(int id)` metodu olduğunu qəbul edirik. `inner` əsl işi görən qiymət xidmətidir. Yeni obyekt eyni müqaviləni saxlayır, çağırışa qeyd əlavə edir və işi inner-ə ötürür. Bu ötürməyə **delegation** deyilir.

Bu **Decorator** pattern-dir. Eyni yanaşma ölçmə və uyğun hallarda keş üçün işlədilə bilər. **Keş** əvvəl alınmış nəticəni təkrar istifadə üçün saxlamaqdır; köhnəlmiş nəticənin nə vaxt silinəcəyi ayrıca seçilir.

Adapter ilə fərq: Adapter müqaviləni uyğunlaşdırır. Decorator eyni müqavilə ətrafında əlavə davranış qurur.

Mediator əlaqələndirməni vasitəçiyə verir

Sifariş yaradılır. Ehtiyat yoxlanmalı, ödəniş təşkil olunmalı, qəbz hazırlanmalıdır. Hər obyekt bütün digərlərini birbaşa tanıyırsa əlaqə şəbəkəsi çətinləşə bilər.

Mediator qarşılıqlı işi əlaqələndirən vasitəçidir. Komponentlər bir-birinin bütün təfərrüatını bilmədən onun üzərindən işləyə bilər.

Sadələşdirilmiş əlaqə

```
Sifaris isteyi
    |
    v
Vasiteci
    +-- uygun isleyiciye yoneldir.
    +-- lazim olan elaqelendirməni aparir.
```

Konkret işi emal edən hissəyə **handler**, işləyici deyilir. .NET-də MediatR adlı kitabxana var, amma pattern-in özü həmin kitabxanadan asılı deyil.

Mühüm sərhəd var: A vasitəçi ilə B-ni, B isə vasitəçi ilə yenə A-nı çağırırsa çağırış dövrəsi yenə qala bilər. Sadəcə araya vasitəçi qoymaq səhv məsuliyyət bölgüsünü avtomatik düzəltmir.

Vasitəçi bütün biznes işlərini özünə yığıb həddindən artıq böyük obyektə də çevrilməməlidir.

Nəticə: Mediator əlaqələri idarə etməyi mərkəzləşdirə bilər. Hər dairəvi asılılığın və hər mürəkkəb sistemin avtomatik həlli deyil.

State Strategy və Template Method fərqi

State cari vəziyyətə görə davranışı dəyişir

Sifariş əvvəl Pending, sonra Paid, sonra Shipped olur. Bunlar gözləmədə, ödənilib və göndərilib vəziyyətləridir. Göndərilmiş sifarişdə bəzi əməliyyatlara artıq icazə verilmir. **State** hər vəziyyətin davranışını ayrıca təşkil edir. Bir vəziyyətdən başqasına keçidə **transition** deyilir.

Strategy-də əsas sual “hansı hesablama üsulunu seçək?” idi. State-də “obyekt hazırda hansı vəziyyətdədir və nə edə bilər?” sualıdır.

Template Method ardıcıllığı sabit saxlayır

Fayl idxalı həmişə oxu, yoxla, çevir, saxla addımlarından keçir. CSV və JSON üçün oxuma addımı fərqlənə bilər. **Template Method** ümumi ardıcıllığı baza sinifdə saxlayır, müəyyən addımları alt sinfə verir. JSON məlumatın açar və dəyərlərlə yazıldığı mətn formatıdır.

Command əməliyyat istəyini obyekt edir

“Sifarişi ləğv et” istəyini əməliyyat növü və sifariş nömrəsi olan obyekt kimi saxlaya bilərik. **Command** həmin istəyi sonradan işlətməyə, növbəyə qoymağa və qeyd etməyə uyğun edir. **Növbə** işlərin sonradan işlənmək üçün saxlandığı ardıcıllıqdır.

Command yaratmaq hər işi geri almağı avtomatik mümkün etmir. Göndərilmiş email üçün sadə undo yoxdur.

Digər struktur patternlərini nümunə ilə ayıraq

Bridge iki ayrı dəyişən istiqaməti ayırır

Bildirişin növü sadə və təcili ola bilər. Göndərmə kanalı email və SMS ola bilər. Hər kombinasiyaya ayrıca sinif yaratmaq əvəzinə bildiriş məntiqi ilə kanalı ayrı saxlayırıq. Bu **Bridge** ideyasıdır. İki tərəf ayrıca genişlənə bilər.

Composite tək elementlə ağacı eyni qayda ilə işlədir

Bir menyu bölməsində məhsullar və başqa alt bölmələr ola bilər. Həm tək məhsula, həm bütöv bölməyə eyni “göstər” əməliyyatı tətbiq etmək istəyirik. **Composite** element və element qrupuna ortaq müqavilə verir. İç-içə qruplar ağac quruluşudur.

Facade mürəkkəb alt sistemi sadə girişlə təqdim edir

Sifariş tamamlamaq üçün bir neçə texniki hissə çağırılmalıdır. **Facade** istifadəçiyə sadə giriş verib həmin təfərrüatları arxada əlaqələndirir. “CompleteOrder” çağırışı bunun nümunəsi ola bilər; transaction zəmanəti ayrıca qurulur.

Flyweight dəyişməz ortaq məlumatı paylaşır

Minlərlə simvol eyni şrift məlumatını ayrıca saxlamaq əvəzinə paylaşa bilər. **Flyweight** belə ortaq hissəni ayırır. Hər simvolun ekrandakı yeri isə ayrıca verilir. Paylaşılan məlumatı nəzarətsiz dəyişmək düzgün deyil.

Proxy obyektə çıxışı idarə edir

Proxy əsl obyektin qarşısında dayanıb giriş icazəsini, uzaq çağırışı və ya gec yaradılmanı idarə edə bilər. Quruluşu Decorator-a bənzəyə bilər. Burada əsas niyyət obyektə çatmağı idarə etməkdir.

Yaradılma və vəziyyət saxlama üçün digər vasitələr

Builder mürəkkəb nəticəni addım-addım qurur

Hesabatın başlığı, bölmələri və filtrləri verilir, sonda Build çağırılır. **Builder** qurulmanı mərhələlərə ayırır və tamamlananda məcburi hissələri yoxlaya bilər. Metodları zəncirlə yazmaq təkbaşına Builder olması demək deyil; əsas niyyət etibarlı nəticə qurmaqdır.

Prototype hazır nümunədən kopya yaradır

Mürəkkəb şablonu sıfırdan qurmaq əvəzinə kopyalaya bilərik. **Prototype** buna xidmət edir. Daxili siyahıları da eyni istinadla paylaşan kopya **shallow copy**-dir. Onları ayrıca kopyalamaq **deep copy** yanaşmasıdır. Hansı hissənin paylaşılacağı düşünülməlidir.

Singleton seçilmiş mühitdə bir nüsxəni paylaşır

Klassik **Singleton** bir nüsxə və ona çıxışı təşkil edir; çox vaxt proqramın hər yerindən əlçatan qlobal girişlə yazılır. DI singleton isə bir container-də paylaşılma qaydasıdır. Bunları eyniləşdirmə. İki prosesin ayrıca nüsxəsi olur. Nüsxənin təhlükəsiz yaradılması onun bütün metodlarının paralel istifadəyə təhlükəsiz olduğunu sübut etmir.

Iterator daxili saxlanmanı açmadan gəzməyə imkan verir

`foreach` siyahının elementlərini gəzir. **Iterator** istifadəçiyə elementləri ardıcıl əldə etməyə imkan verir; onların daxildə massiv, ağac və ya başqa quruluşda olması gizli qala bilər.

Memento əvvəlki vəziyyəti saxlayır

Redaktorda son dəyişikliyi geri almaq üçün əvvəlki vəziyyətin surətini saxlayırıq. **Memento** bunu obyektin daxili quruluşunu hər yerə açmadan təşkil edir. Böyük surətlərin yaddaş xərci vardır.

Hadisə zəncir və qayda emalı

Observer dəyişiklik barədə abunə olanlara xəbər verir

Qiymət dəyişəndə ekran və başqa izləyicilər yenilənsin. **Observer** hadisəni yayan obyektlə həmin hadisəyə abunə olanları əlaqələndirir. C# event mexanizmi bu ideyaya nümunədir. Uzunömürlü yayıcı artıq lazım olmayan abunəçini saxlaya bilər; abunəliyin bitirilməsi nəzərə alınır.

Chain of Responsibility sorğunu işləyicilərdən keçirir

Sorğu A, B və C mərhələlərindən keçir. B işi tamamlayıb dayandıra və ya C-yə ötürə bilər. **Chain of Responsibility** bu quruluşdur. Middleware dərsində gördüyümüz axın buna yaxın nümunədir. Sıra dəyişəndə nəticə də dəyişə bilər.

Interpreter kiçik bir dilin qaydalarını işlədir

`total > 100 AND vip` kimi sadə qaydanın mənasını müəyyən edib qiymətləndiririk. **Interpreter** həmin dilin hissələrini təmsil edir və şərh edir. **Grammar** hansı ifadələrin düzgün qurulduğunu bildirən qaydalardır. Çox böyük dil üçün bu quruluş ağırlaşa bilər.

Visitor sabit obyekt quruluşuna yeni əməliyyat verir

Məlumat ağacındakı tiplər sabitdir, amma onları çap etmək, hesablamaq və yoxlamaq kimi əməliyyatlar artır. **Visitor** əməliyyatı həmin tiplərdən ayrı təşkil edir. Yeni əməliyyat əlavə etmək rahatlaşa, yeni obyekt tipi əlavə etmək isə bütün visitor-lərə təsir edə bilər.

Nəticə: bu pattern-lərin hamısı “kod səliqəli olsun” deyə eyni işi görmür. Hərəsi konkret dəyişmə problemini başqa yerdə saxlayır.

Backend-də eşidəcəyin əlavə patternlər

Bunlar klassik 23 GoF pattern ilə eyni siyahı deyil, amma müsahibədə tez-tez yanaşı çəkilir.

Repository domain obyektlərinə çıxışı bir sərhəddə toplayır. Domain biznes anlayışları və qaydalarıdır. “Müştərinin aktiv sifarişlərini ver” əməliyyatı saxlanma təfərrüatından ayrılmış müqavilə ola bilər.

Unit of Work bir iş ərzində dəyişən obyektləri izləyib yazmaları əlaqələndirir. EF Core DbContext bu imkanların çoxunu artıq verir. Üzərinə əlavə ümumi repository yazmaq hər layihədə məcburi deyil.

CQRS dəyişiklik əmrləri ilə oxuma sorğularının məsuliyyətini və modelini ayırır. Ayrı baza və ayrı mikroservis məcburi deyil. Əsas fikir yazma modeli ilə oxuma ehtiyacının həmişə eyni formaya sıxışdırılmamasıdır.

Outbox sifariş dəyişikliyi ilə göndəriləcək hadisəni eyni baza transaction-unda saxlayır. Ayrı işçi sonradan hadisəni göndərir. Bu, “sifariş yazıldı, xəbər itdi” problemini həll etməyə kömək edir. Amma hadisə təkrar göndərilə bilər.

Idempotency eyni iş təkrar gələndə əlavə yanlış təsir yaratmamaqdır. Məsələn, eyni hadisə nömrəsi ilə iki dəfə ödəniş yazmamaq. “SaveChanges et, sonra Publish et” təkbaşına iki ayrı sistem arasında birlikdəlik zəmanəti yaratmır. Publish hadisəni yaymaq deməkdir.

Adı deyil problemi tanımaq üçün məşq

Problem	Düşünüləcək pattern
Yeni endirim hesabı əlavə olunur	Strategy
Uyğun qəbz obyektini bir yerdən almaq lazımdır	Factory yanaşması
Köhnə printerin metodu yeni müqaviləyə uyğun deyil	Adapter
Əsas qiymət koduna toxunmadan qeyd əlavə edilir	Decorator
Obyektin vəziyyəti hansı əməliyyatın mümkün olduğunu dəyişir	State
Mürəkkəb alt sistemə sadə giriş lazımdır	Facade

Özünü yoxla

“Adapter ilə Decorator eyni deyil” cümləsinin səbəbini de. **Adapter fərqli müqaviləni uyğunlaşdırır; Decorator eyni müqavilə ətrafına davranış əlavə edir.**

“Strategy ilə Factory fərqi nədir?” sualına cavab ver. **Biri dəyişən davranışın icrasını ayırır, o biri uyğun obyektin alınmasını təşkil edir.**

Yeni pattern istifadə edəndə onun çıxarılması ilə hansı konkret problemin geri qayıdacağını izah edə bil. Cavab yalnız “kod çirkin olar” olmamalıdır.

Əsas nəticə: pattern seçimi problemin yeri və dəyişən hissəsi ilə əsaslandırılır. Adı əzbərləmək həmin qərarı əvəz etmir.

9 Async gözləyərkən threadi boş saxlamağa kömək edir

Kafe tətbiqi uzaq ödəniş xidmətindən cavab gözləyir. Cavab gələnə qədər proqramın həmin yerdə hesablayacağı iş yoxdur. Sadəcə başqa sistemin işini bitirməsi lazımdır.

İki yanaşma ola bilər. Kod icra edən axını gözləmədə tuturuq. Yaxud cavab gələndə davam etmək üçün məlumatı saxlayıb axını başqa işə buraxırıq.

Thread prosesdə kod icra edən axındır. **Asinxron gözləmə** müəyyən əməliyyatın tamamlanmasını gözləyərkən həmin thread-i bloklamamağa imkan verir.

Gözləmə fərqi

```
Bloklayan gozleme:
    Thread bu yerde gozleyir, basqa is gormur.

Asinxron gozleme:
    Davamin haradan olacagi saxlanir.
    Cavab hazir olanda davam kodu isleyir.
```

Bu, uzaq xidmətin öz işini avtomatik sürətləndirmir. Cavab yenə vaxtında gəlir. Qazanc odur ki, gözləmə zamanı thread başqa iş üçün əlçatan qala bilər.

Bir əməliyyatın cavab müddəti ilə serverin eyni müddətdə neçə işə xidmət etməsi ayrı göstəricilərdir. Birincisinə latency, ikincisinə throughput deyilir.

Task işi təmsil edir amma threadin özü deyil

Ödəniş başladılır. Dərhal yekun cavab yoxdur, amma onun tamamlanmasını təmsil edən obyekt alırıq. .NET-də belə tamamlanma üçün **Task** istifadə olunur.

`Task` nəticə dəyəri olmayan tamamlanmanı, `Task<int>` isə tamamlananda tam ədəd verən işi təmsil edə bilər. `<int>` nəticənin tipini göstərir.

Task barədə soruşduğumuz məlumat

```
Is bitibmi?
Ugurlu bitibse netice nedir?
Xeta ile bitibmi?
Legv olunubmu?
```

Task-ın olması bütün gözləmə müddətində ayrıca thread tutulması demək deyil. Şəbəkə cavabını əməliyyat sistemi izləyib hazır olanda xəbər verə bilər.

ThreadPool .NET-in qısa işlər üçün idarə etdiyi işçi thread-lər mexanizmidir. `Task.Run` işi adətən həmin pool-da planlayır. Amma bütün Task-lar Task.Run işi deyil. Artıq tamamlanmış Task və xarici cavabı gözləyən Task da mümkündür.

Yadda saxla: Task “bu əməliyyatın tamamlanmasını və nəticəsini izləyə bilərəm” deməkdir. “Bu əməliyyatın həmişə ayrıca bir thread-i var” demək deyil.

Async və await işləyəndə kod hansı sırada gedir

Bu kiçik C# nümunəsini addım-addım oxuyaq:

C#

```
static async Task DemoAsync()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}

Task work = DemoAsync();
Console.WriteLine("C");
await work;
```

Əvvəl DemoAsync çağırılır və **A** yazılır. `Task.Delay(1000)` təxminən 1000 millisaniyə sonra tamamlanan gözləmə yaradır. Bir saniyə 1000 millisaniyədir.

Delay hələ bitmədiyi üçün `await` yerində metodun davamı saxlanır və çağıran kod davam edə bilir. Ona görə **C** yazılır. Delay tamamlananda metod qaldığı yerdən davam edir və **B** yazılır. Gözlənən sıra **A, C, B**-dir.

`async` metodda belə gözləmələr yazmağa imkan verir. Metod çağırılan kimi ilk tamamlanmamış await-ə qədər adi qaydada işləyir. **Async yazmaq kodu əvvəldən avtomatik başqa thread-ə keçirmir.**

Əgər await olunan Task artıq bitibsə metod dayanmadan davam edə bilər. Hər await mütləq başqa thread-ə keçid demək deyil.

Davam harada saxlanır və I O ilə CPU niyə fərqlidir

Metod gözləyəndə hara qayıdacağını və lazım olan dəyərləri saxlamalıdır. Kompilyator, C# mətnini işləyən formaya çevirən alət, bunun üçün **state machine** yaradır. Bu, metodun hansı mərhələdə olduğunu saxlayan mexanizmdir. Gözləmədən sonrakı kod **continuation**, davam kodu adlanır.

İndi işin əsas vaxtının hara getdiyini ayıraq:

İşin növü	Əsas vaxt nəyə gedir
I/O-bound	Şəbəkə, disk və ya baza cavabını gözləməyə.
CPU-bound	Prosessorda hesablama aparmağa.

I/O məlumatın proqramdan xaricə çıxması və ya xaricdən gəlməsidir. **CPU** hesablamaları yerinə yetirən prosessordur.

Şəkli serverdən almaq I/O üstünlüklü, onun üzərində ağır hesablama etmək CPU üstünlüklü ola bilər.

C# istifadə fraqmentləri

```
// Hazir asinxron I/O emeliyyatini gozle:
string text = await http.GetStringAsync(url);

// Lazim olanda CPU isini pool-da planla:
int value = await Task.Run(() => Calculate());
```

`http` HttpClient obyektidir, `url` ünvan, `Calculate` hesablayan metodudur. Sinxron baza çağırışını Task.Run ilə sarımaq onu həqiqi asinxron baza API-sinə çevirmir. Serverdə əlavə Task.Run ümumi hesablama gücünü artırmır.

WhenAll müstəqil gözləmələri üst üstə salır

Qiymət sorğusu 300, ehtiyat sorğusu 500 millisaniyə gözləyir. Biri o birinin nəticəsindən asılı deyil.

C# ardıcıl variant

```
var price = await GetPriceAsync();
var stock = await GetStockAsync();
```

İkinci iş birinci bitəndən sonra başlayır. Sadə modeldə gözləmə `300 + 500 = 800` millisaniyədir.

İndi əvvəl hər ikisini başladaq:

C# müstəqil işlər üçün variant

```
var priceTask = GetPriceAsync();
var stockTask = GetStockAsync();

await Task.WhenAll(priceTask, stockTask);

var price = await priceTask;
var stock = await stockTask;
```

`GetPriceAsync` qiyməti, `GetStockAsync` ehtiyatı əldə edən nümunə metod adlarıdır. Onların əvvəl çağırılması işləri başladır. **WhenAll** verilmiş bütün Task-lar tamamlananda tamamlanan Task verir. Onları avtomatik ayrıca CPU nüvələrində işlətmək vədi vermir.

İdeal sadə gözləmə modelində ümumi müddət təxminən ən uzun gözləməyə, 500 millisaniyəyə yaxın ola bilər. Həqiqi vaxt xarici xidmətlərdən və limitlərdən asılıdır.

Şərt: işlər müstəqil olmalıdır. Eyni DbContext-də iki paralel baza əməliyyatını bu cür işlətmək olmaz. Eyni context eyni anda bir neçə əməliyyata uyğun deyil.

Concurrency ilə parallelism fərqi

Concurrency bir neçə işin vaxt baxımından üst-üstə irəliləməsidir. Qiymət cavabını gözləyərkən ehtiyat sorğusunu başlatmaq bunun nümunəsidir. Hər an bir thread-də kod işləməsi də mümkün ola bilər.

Parallelism iş hissələrinin həqiqətən eyni anda müxtəlif icra resurslarında hesablanmasıdır. Bir neçə prosessor nüvəsində müxtəlif şəkillərin emalı nümunədir.

İki müxtəlif fikir

```
Concurrency:
    Iki xarici cavabi eyni vaxt araliginda gozleyirik.

Parallelism:
    Iki hesablamani heqiqeten eyni anda icra edirik.
```

Async əsasən gözləmə ilə işləmə üsuludur. Paralellik isə eyni anda hesablama aparmağa aiddir. Onlar birlikdə istifadə edilə bilər, amma bir-birinin adı deyil.

Hər işi limitsiz eyni anda başlatmaq da düzgün deyil. 50 min sorğunu birdən xarici xidmətə göndərmək həmin xidməti və bağlantı ehtiyatını tükədə bilər.

Buna görə bəzi işlərdə maksimum paralel əməliyyat sayı seçilir. Məsələn, eyni anda yalnız 4 fayl endirilsin. Dörd rəqəmi universal düzgün rəqəm deyil; xidmətin limiti və resurslar üzrə seçilir.

Yadda saxla: “bir neçə iş başladı” deyəndə onların gözləmələri üst-üstə düşür, yoxsa hesablamaları həqiqətən paralel işləyir, bunu dəqiqləşdir.

Xəta olanda WhenAll və WhenAny nə edir

Bir Task xəta ilə bitə bilər. Xəta obyektinə **exception** deyilir. Onu await edəndə xəta çağıran koda ötürülə bilər.

C#

```
try
{
    await Task.WhenAll(priceTask, stockTask);
}
catch (Exception error)
{
    Console.WriteLine(error.Message);
}
```

`try` yoxlanılan iş blokudur. `catch` uyğun xətanı tutur. `error.Message` xətanın mesajıdır.

WhenAll işlərin hamısının tamamlanmasını gözləyir. Biri xəta verəndə qalanları avtomatik dayandırmır. Ümumi Task bütün xətaları öz Exception məlumatında saxlaya bilər; await isə adətən həmin xətalardan birini yenidən atır. Hamısı lazımdırsa ümumi Task-ın xəta toplusuna baxılır.

WhenAny ilk tamamlanan Task-ı verir. Tamamlanma uğur, xəta və ya ləğv ola bilər. Verilən Task-ı ayrıca await etmək onun həqiqi nəticəsini və ya xətasını almaq üçündür.

WhenAny də qalan işləri avtomatik ləğv etmir. “Birincisi hazırdır” ilə “o biriləri dayandırıldı” ayrı hadisələrdir.

Əsas nəticə: tamamlanmanı izləmək, xətanı almaq və digər işləri ləğv etmək ayrı qərarlardır.

CancellationToken işi zorla öldürmür

Müştəri gözləməkdən vaz keçir. Və ya iş üçün ayrılmış 5 saniyə bitir. Davam etmək istəmədiyimizi əməliyyata bildirmək lazımdır.

CancellationToken ləğv istəyi barədə siqnalı daşıyır. **CancellationTokenSource** həmin siqnalı yaradan və ləğv edən obyektdir.

C#

```
using var stop = new CancellationTokenSource();
stop.CancelAfter(TimeSpan.FromSeconds(5));

string text = await http.GetStringAsync(url, stop.Token);
```

`CancelAfter` müddət bitəndə ləğv istəyir. `TimeSpan` vaxt müddətini bildirir. Token http metoduna ötürülür ki, həmin əməliyyat siqnalı nəzərə ala bilsin. `using` iş bitəndə source-un resurslarını təmizləyir.

Öz uzun dövrümüzdə də siqnalı vaxtaşırı yoxlaya bilərik. `ThrowIfCancellationRequested()` istək varsa ləğv xətası verir. Buna əməkdaşlığa əsaslanan ləğv deyilir: kod siqnalı qəbul edir və dayanır.

Timeout müəyyən vaxtdan artıq gözləməməkdir. Gözləmənin bitməsi işin özünün mütləq dayandığını göstərmir. Əməliyyatın dayanması lazımdırsa onun cancellation dəstəyi və token-in ona ötürülməsi əhəmiyyətlidir.

İstifadəçi ləğvi ilə vaxt sərhədini birləşdirən linked token də yaratmaq olar: hər iki mənbədən biri ləğv istəyəndə ümumi siqnal verilir.

Result və Wait niyə problem yarada bilər

`.Result` və `.Wait()` Task bitənə qədər çağıran thread-i bloklaya bilər. UI proqramında bu ekranı dondura bilər. **UI** istifadəçi interfeysidir; onun xüsusi thread-i düymə və ekran yeniləmələrini idarə edir.

Dairəvi gözləmə nümunəsi

```
UI thread: ".Result ile metodun bitmesini gozleyirem."

Metodun davami: "Bitmek ucun UI thread-de islemeliyem."

UI neticeni, netice ise UI thread-i gozleyir.
```

Belə irəliləyə bilməyən qarşılıqlı gözləmə **deadlock** adlanır.

SynchronizationContext await-dən sonra davamın hansı mühitdə işləməsini bildirə bilən mexanizmdir. UI-də bu çox vaxt UI thread-idir. `ConfigureAwait(false)` həmin await üçün bu mühitə qayıtmaq tələbini azaltmağa imkan verir. “Mütləq yeni thread aç” və ya “bütün deadlock-ları düzəlt” demək deyil.

ASP.NET Core-da standart klassik request SynchronizationContext yoxdur. Amma bloklama yenə çoxlu thread-i gözləmədə tutub başqa işlər üçün thread çatışmazlığı yarada bilər. Buna **ThreadPool starvation** deyilir.

İlkin seçim: asinxron işi mümkün olduqca çağırış zənciri boyunca await et. Gözləmə problemini `.Result` ilə yuxarıya gizlətmə.

Async void və nəzarətsiz fon işi

`async Task` metodu çağırana tamamlanmanı izləmək üçün Task verir. `async void` isə belə Task vermir. Ona görə onu adi metod kimi await edib tamamlanmasını və xətasını izləmək olmur.

Əsas istisna void imzası tələb edən hadisə işləyicisidir. Məsələn, UI düyməsinin click handler-i. Handler hadisə gələndə çağırılan koddur. Adi iş metodları üçün Task və ya Task nəticəsi olan variant seçilir.

Fire and forget işi başladıb sonunu izləməməkdir. HTTP sorğusunda işi belə başlatmaq onun etibarlı bitməsinə zəmanət vermir.

Ömür problemi

```
Sorqu baslayir, DbContext alir.
    |
    +-- gozlenilmeyen fon isi hemin context-i tutur.

Sorqu bitir, scope temizlenir.
Fon isi hele context-den istifade etmek isteyir.
```

DI dərsində scope-un xidmətləri hansı sərhədə qədər saxladığını gördük. Fon işi sorğudan uzun yaşayırsa onun resursları da ayrıca idarə olunmalıdır.

Etibarlı fon emalı üçün tətbiqin idarə etdiyi worker, öz scope-u və iş itməməlidirsə davamlı növbə istifadə oluna bilər. **Worker** arxa planda işi emal edən hissədir. Davamlı növbə isə proqram bağlansa da işi saxlayan mexanizmdir.

Resurs await bitənə qədər açıq qalmalıdır

Fayldan mətn oxuyuruq. Reader obyektini oxuma tamamlanmamış bağlasaq başladılmış iş artıq bağlanmış resursla qarşılaşa bilər.

C#

```
static async Task<string> ReadAsync(string path)
{
    using var reader = new StreamReader(path);
    return await reader.ReadToEndAsync();
}
```

`StreamReader` fayldan mətn oxuyan obyektdir. `path` faylın yoludur. `ReadToEndAsync` sonadək oxumağı təmsil edən Task verir. Await onun bitməsini gözləyir, sonra metoddan çıxırıq və using reader-i təmizləyir.

Burada await-i düşünmədən çıxarıb hələ tamamlanmamış Task-ı qaytarmaq resursun tez bağlanmasına səbəb ola bilər. **Kodun qısalması həmişə eyni davranışı qorumaq demək deyil.**

İndi **ValueTask** adını da yerinə qoyaq. O, bəzi hallarda hazır nəticəni ayrıca Task yaratmadan təmsil edə bilər. Çox tez-tez dərhal tamamlanan və ölçülmüş xərcin vacib olduğu əməliyyatlarda faydalı ola bilər.

ValueTask həmişə stack-də saxlanır demək deyil. **Stack** çağırışlara aid məlumatın saxlandığı yaddaş sahəsidir. ValueTask başqa obyektin sahəsində də yerləşə bilər; yer onun harada istifadə edilməsindən asılıdır. Ümumi qayda kimi onu bir dəfə await et. Dəfələrlə gözləmək lazımdırsa Task müqaviləsi daha rahatdır. Sadə tətbiq metodlarında başlanğıc seçim çox vaxt Task-dır.

Eyni məlumatı paralel dəyişəndə nəticə itə bilər

Sayğac 0-dır. İki thread `counter++` edir. Hər iki artım saxlanarsa nəticə 2 olmalıdır. Amma ++ oxu, artır, yaz addımlarından ibarətdir:

Addım	Thread A	Thread B
Oxuma	0 oxuyur	0 oxuyur
Hesablama	1 hesablayır	1 hesablayır
Yazma	1 yazır	1 yazır

Son nəticə 1 qala bilər. Nəticənin nəzarətsiz icra sırasından asılı olmasına **race condition** deyilir.

`Interlocked.Increment(ref counter)` artımı digər paralel artımlara qarşı bölünməz əməliyyat edir. **Atomic** burada həmin addımın araya qarışdırılmadan yerinə yetirilməsi deməkdir. `ref` dəyişənin özünə istinad ötürülməsini bildirir.

Lock müəyyən kod hissəsinə eyni anda girişi məhdudlaşdırır. Adi lock bloku daxilində await istifadə olunmur. Asinxron gözləmə ilə giriş limiti üçün `SemaphoreSlim.WaitAsync` uyğun ola bilər. Semaphore müəyyən sayda icazə saxlayır; iş bitəndə icazə geri verilir.

Nəticə: async yazmaq paylaşılan məlumatı avtomatik təhlükəsiz etmir. Bir işi gözləmək ilə eyni məlumata paralel girişi idarə etmək ayrı məsələlərdir.

Bütün dərsin məntiqini bir işə tətbiq et

Kafe ekranı qiymət və ehtiyat istəyir. Bu iki xarici sorğu bir-birindən asılı deyilsə əvvəl ikisini başlatmaq, sonra WhenAll ilə nəticələri gözləmək olar. Eyni DbContext-i paralel paylaşmaq isə olmaz.

İstifadəçi ekranı bağlayanda ləğv token-i əməliyyatlara ötürülür. Token dayanma istəyidir; hər işi zorla öldürən mexanizm deyil.

Nəticə alınanda UI görünüşü yenilənir. UI tələb olunan kod həmin mühitin thread qaydasına əməl etməlidir. Eyni thread-ə həmişə qayıdacağına və ya hər await-də thread-in dəyişəcəyinə dair ümumi hökm yoxdur.

Özünü yoxla

Task ayrıca thread-dirmi? Xeyr. Əməliyyatın tamamlanmasını və nəticəsini təmsil edir.

Async CPU hesabını avtomatik sürətləndirirmi? Xeyr. Gözləməni bloklamadan idarə etməyə imkan verir; CPU işi üçün alqoritm və hesablama resursları ayrıca məsələdir.

WhenAll-da biri xəta atsa digərləri dayanırmı? Avtomatik dayanmır. Onların ləğvi ayrıca idarə olunur.

Bir cümləlik məntiq: işi başladırıq, tamamlanmanı Task ilə təmsil edirik, lazım olan nəticəni await ilə gözləyirik və işin resurs, xəta, ləğv sərhədlərini ayrıca düzgün saxlayırıq.